# Course 3: Unsupervised Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

# Summary

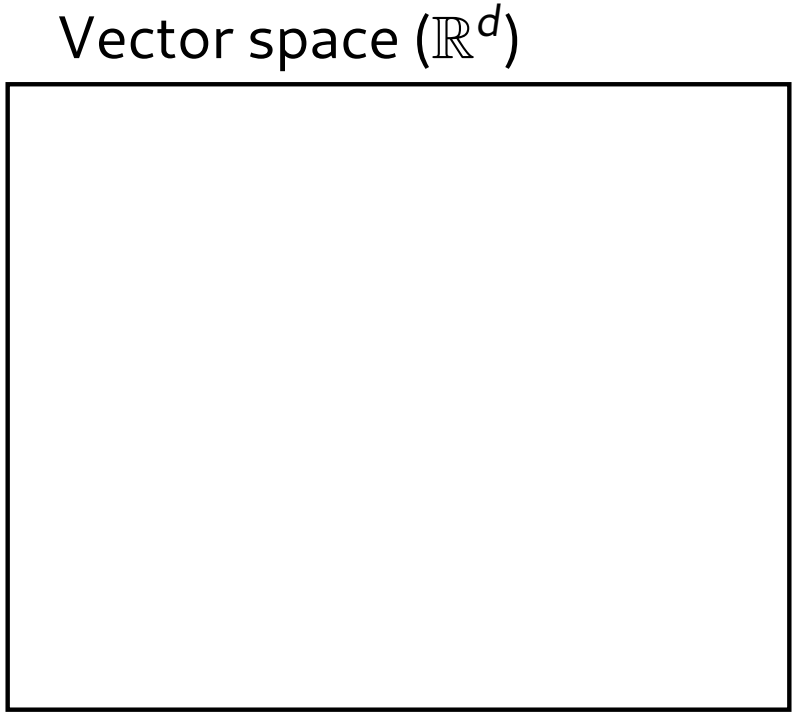
**Last session**

1. Supervised learning - learning from labeled examples
2. Challenges of supervised learning
3. Evaluation: ground truth and metrics

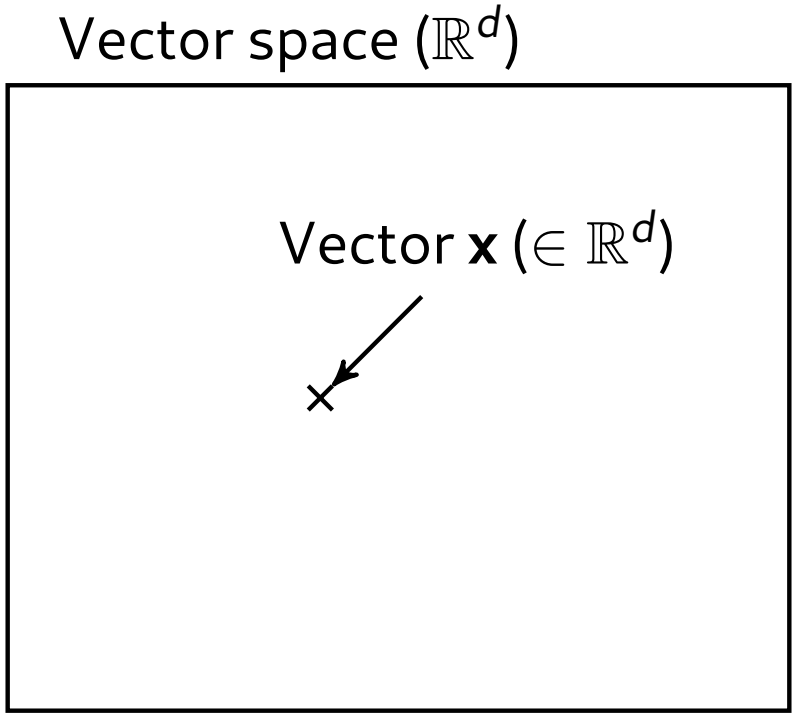
**Today's session**

1. Learning from unlabeled examples
   1. Clustering
   2. Dimensionality reduction (PCA, t-SNE)
2. Evaluating without labels
3. A note on data cleaning

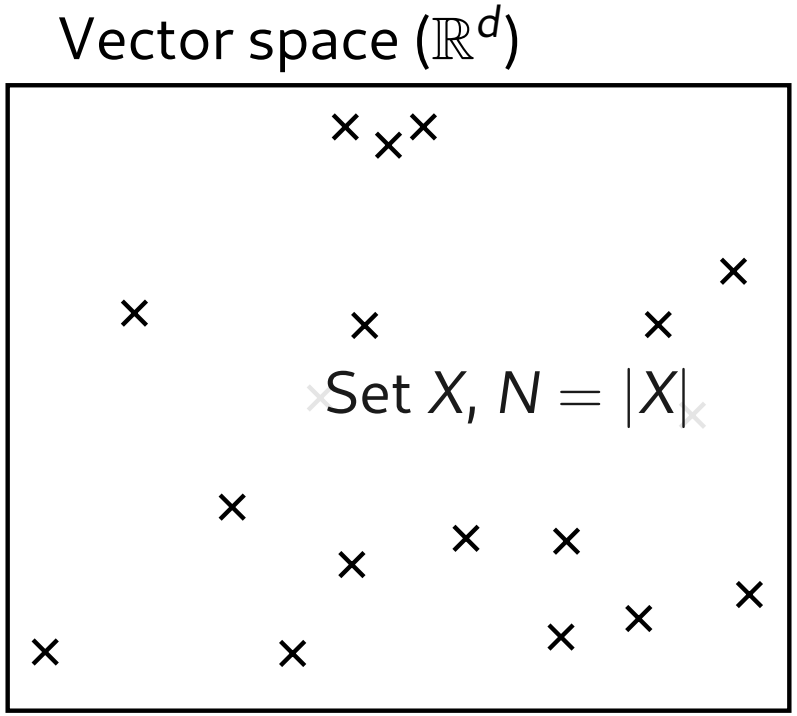
# Notations



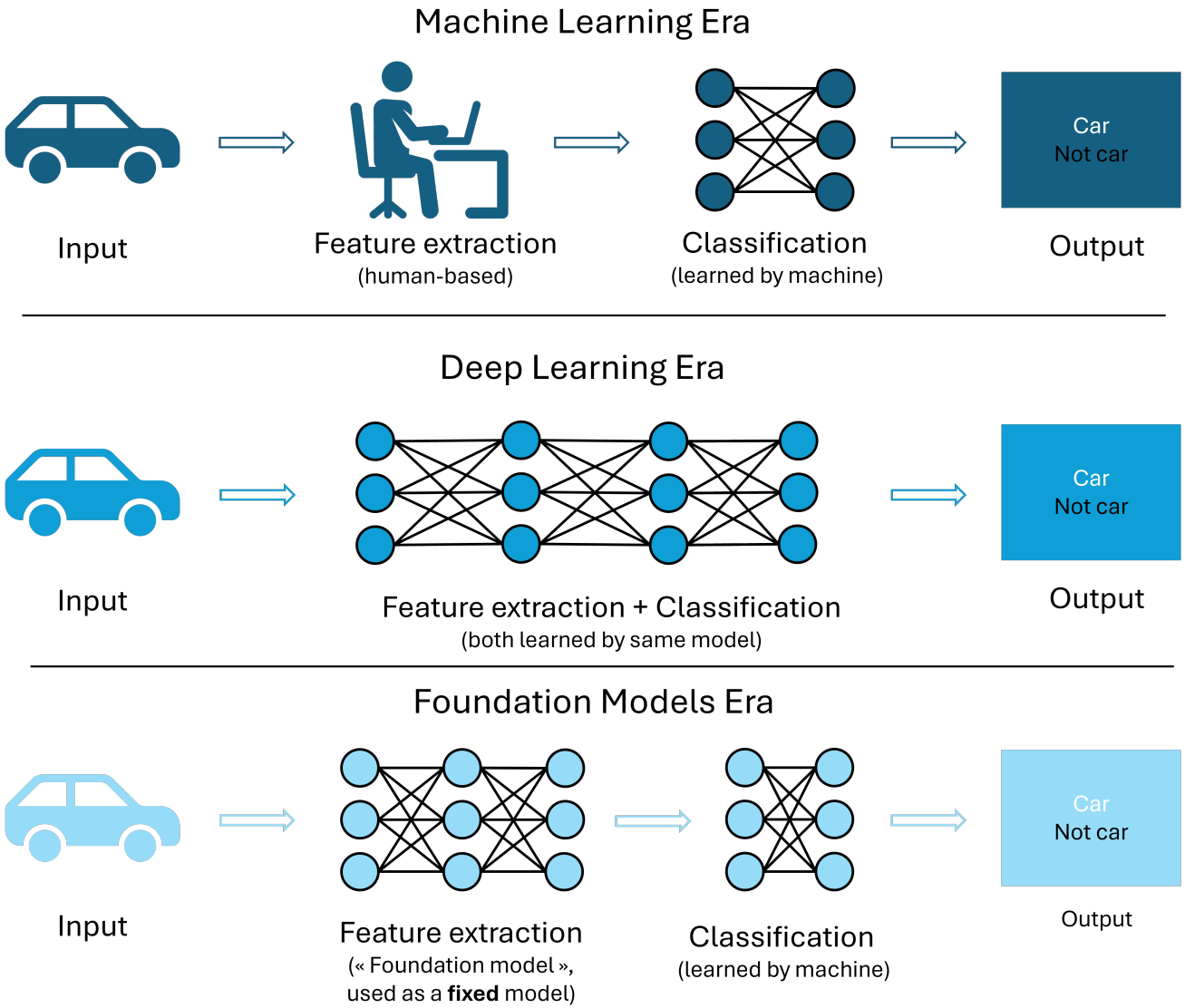
# Notations



# Notations



# What is the vector $x$?




(Similar to last session)

# Unsupervised learning

## Goal

Discover patterns/structure in $X$,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point. $\rightarrow$ Useful to visualize data!

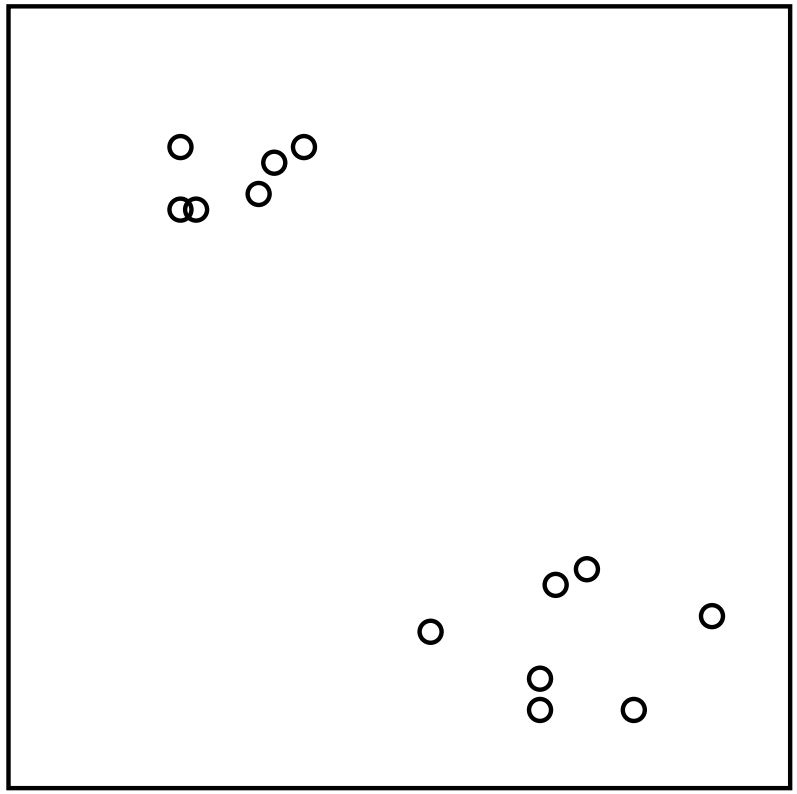
# Unsupervised learning

## Goal

Discover patterns/structure in $X$,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point.
    $\rightarrow$ Useful to visualize data!



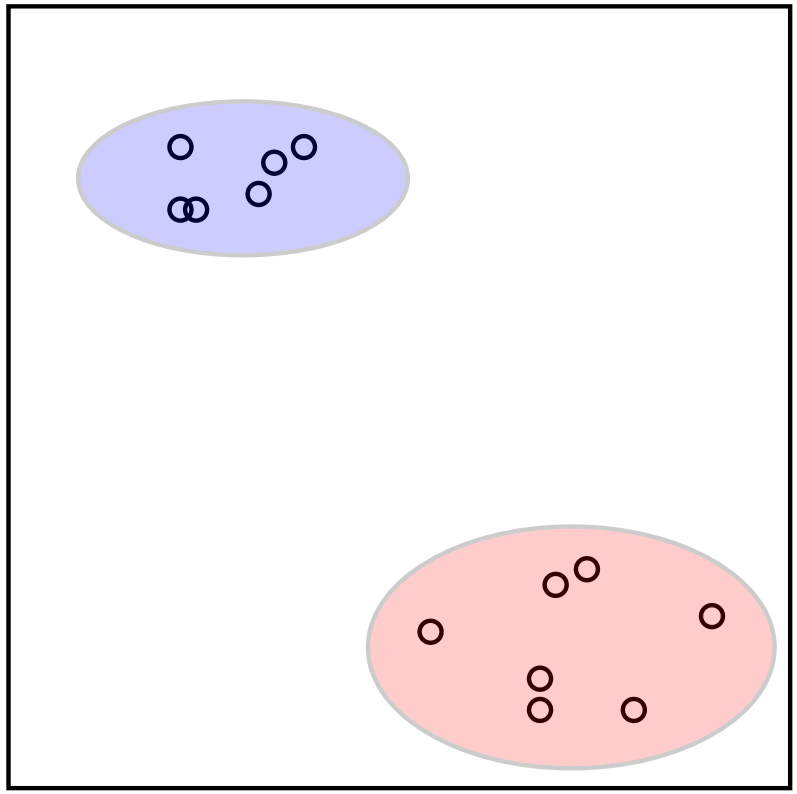
# Unsupervised learning

## Goal

Discover patterns/structure in $X$,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point.
    $\rightarrow$ Useful to visualize data!



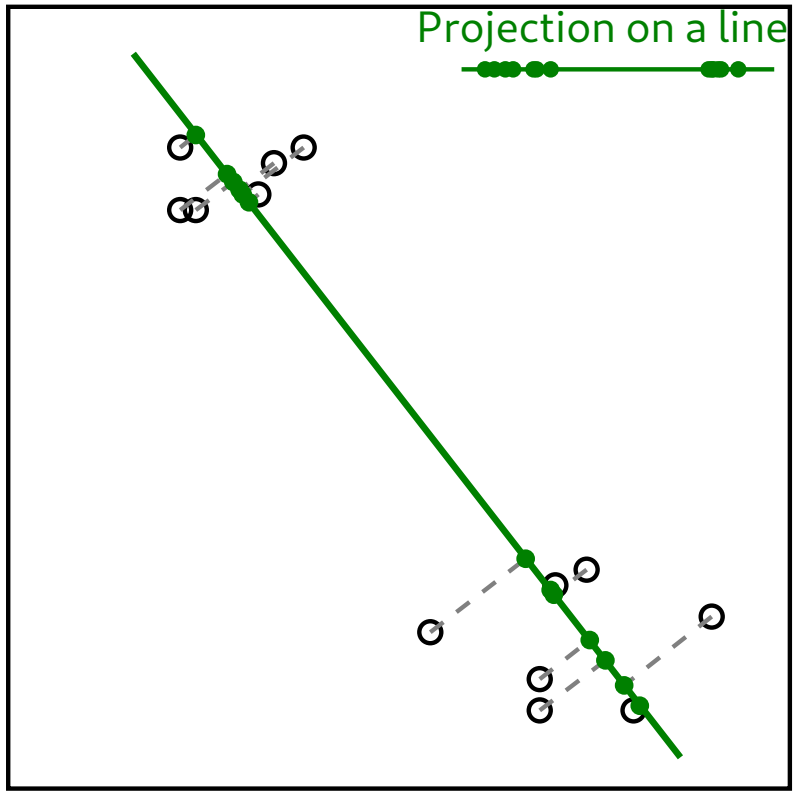
# Unsupervised learning

## Goal

Discover patterns/structure in $X$,

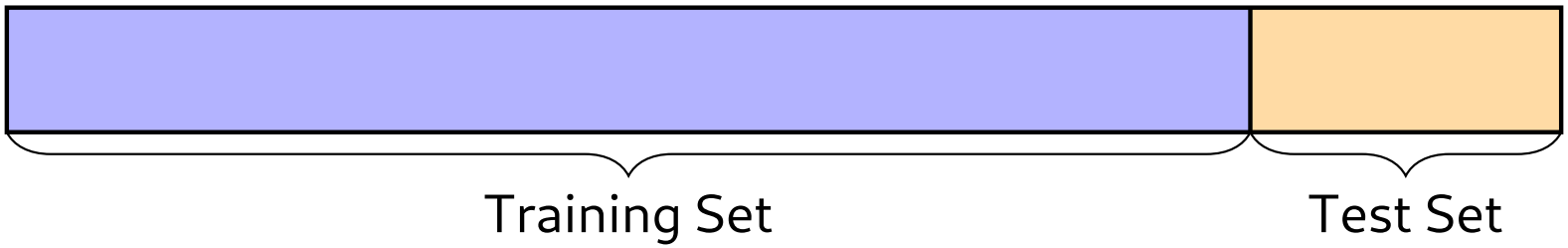
## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering: group similar points,
  - Dimensionality reduction: fewer coordinates per point.
    $\rightarrow$ Useful to visualize data!

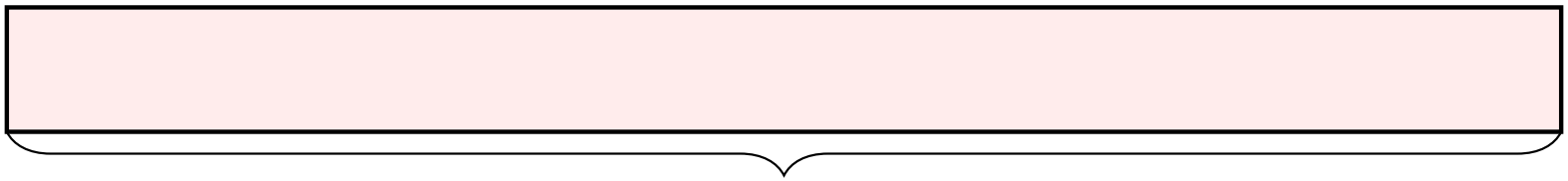


# Training and Test sets?

Full Dataset in train/test sets (as for supervised):




...Or actually:

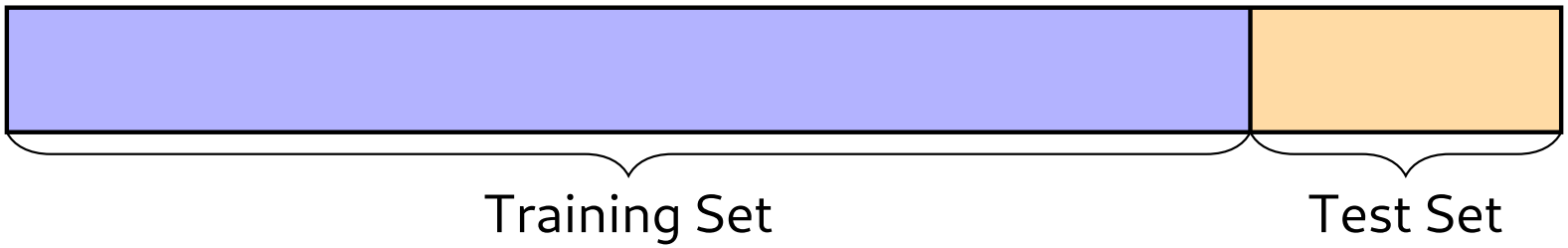



**The same training and test sets??**

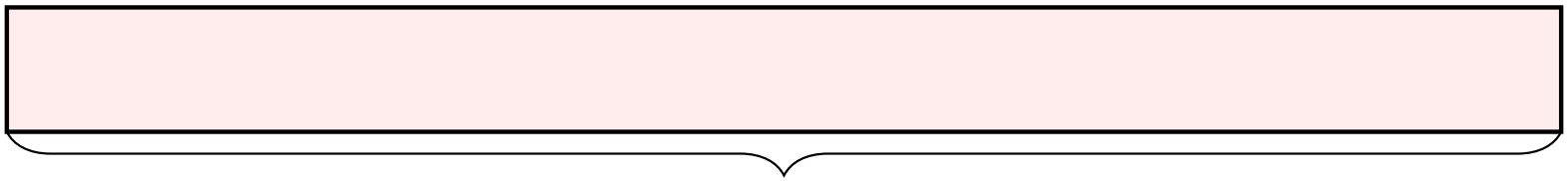
Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant **(there is no "correct answer" to learn and generalize!)**.

# Training and Test sets?

Full Dataset in train/test sets (as for supervised):




...Or actually:



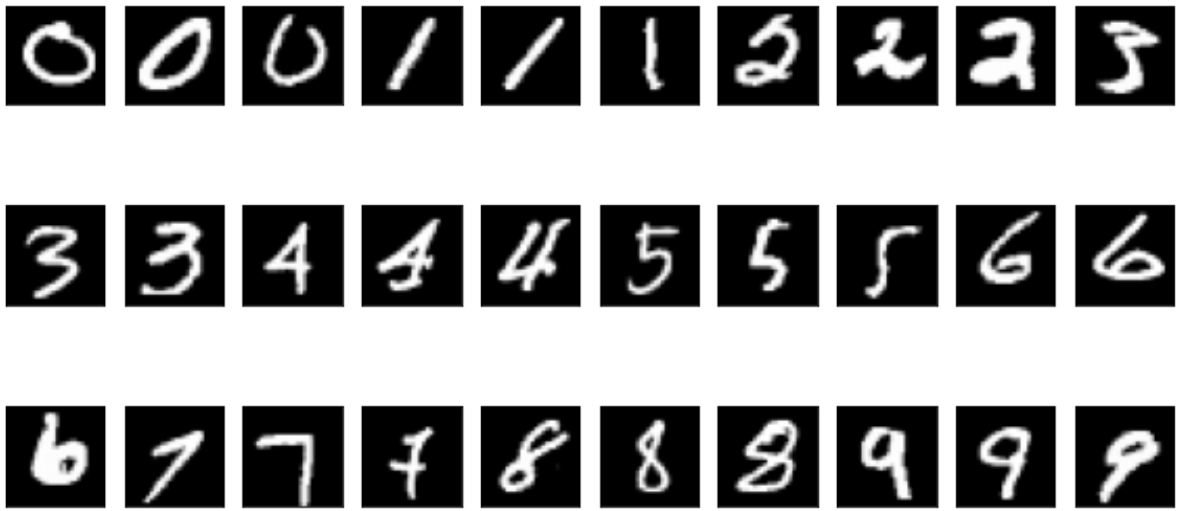

## The same training and test sets??

Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant **(there is no "correct answer" to learn and generalize!)**.

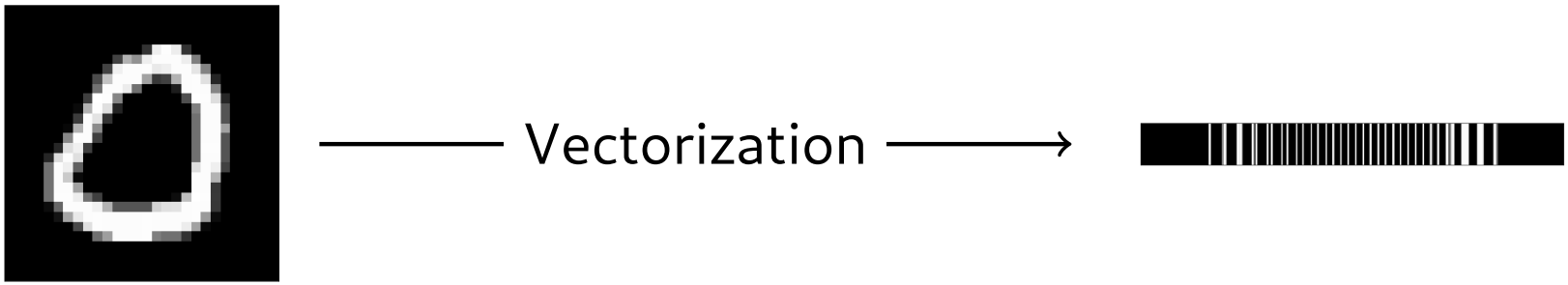
# A classical dataset: MNIST dataset (1/2)

## MNIST Dataset

- "Toy" dataset (=small and easy)
- 60000 + 10000 handwritten digits



# A classical dataset: MNIST dataset (2/2)




Hence, all images are interpreted as 1D vectors!

# Plan

1 Clustering

2 Dimensionality reduction

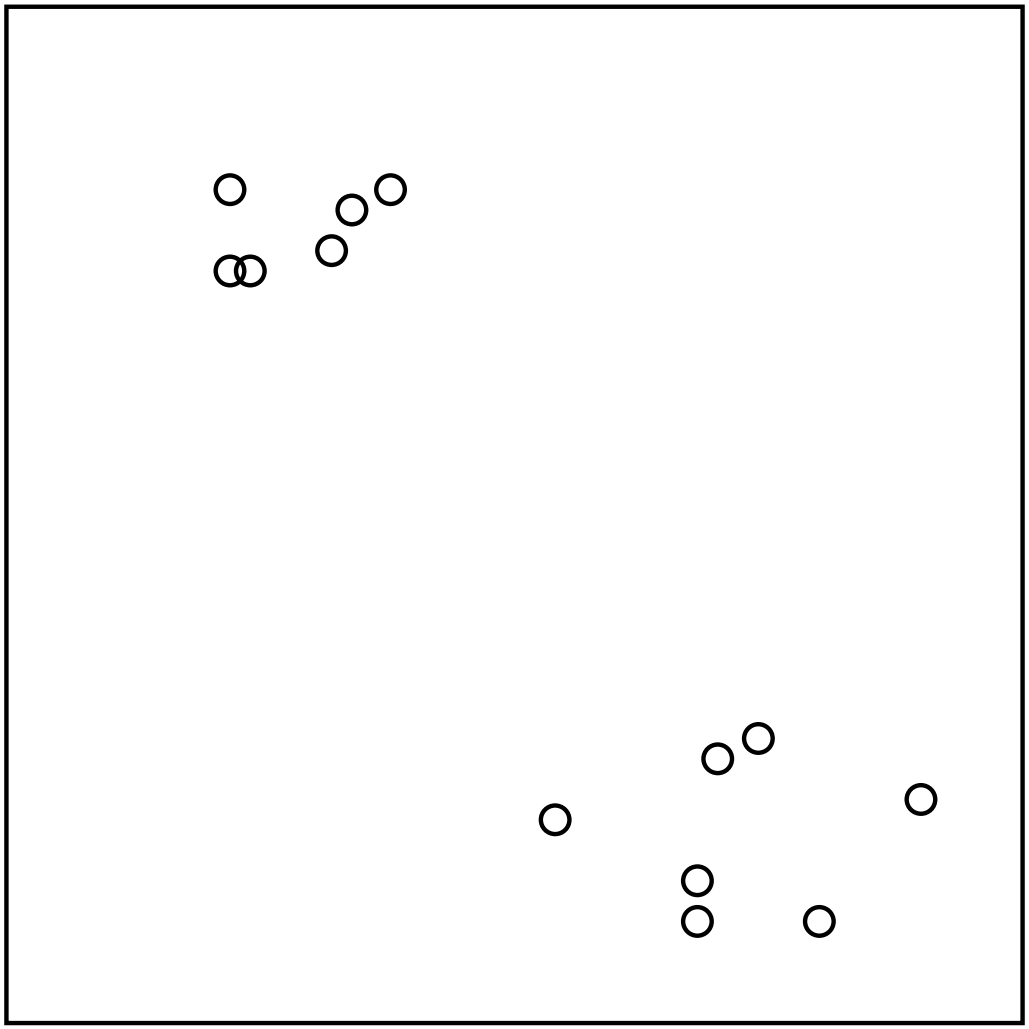
3 Evaluating unsupervised models

4 A note on data cleaning

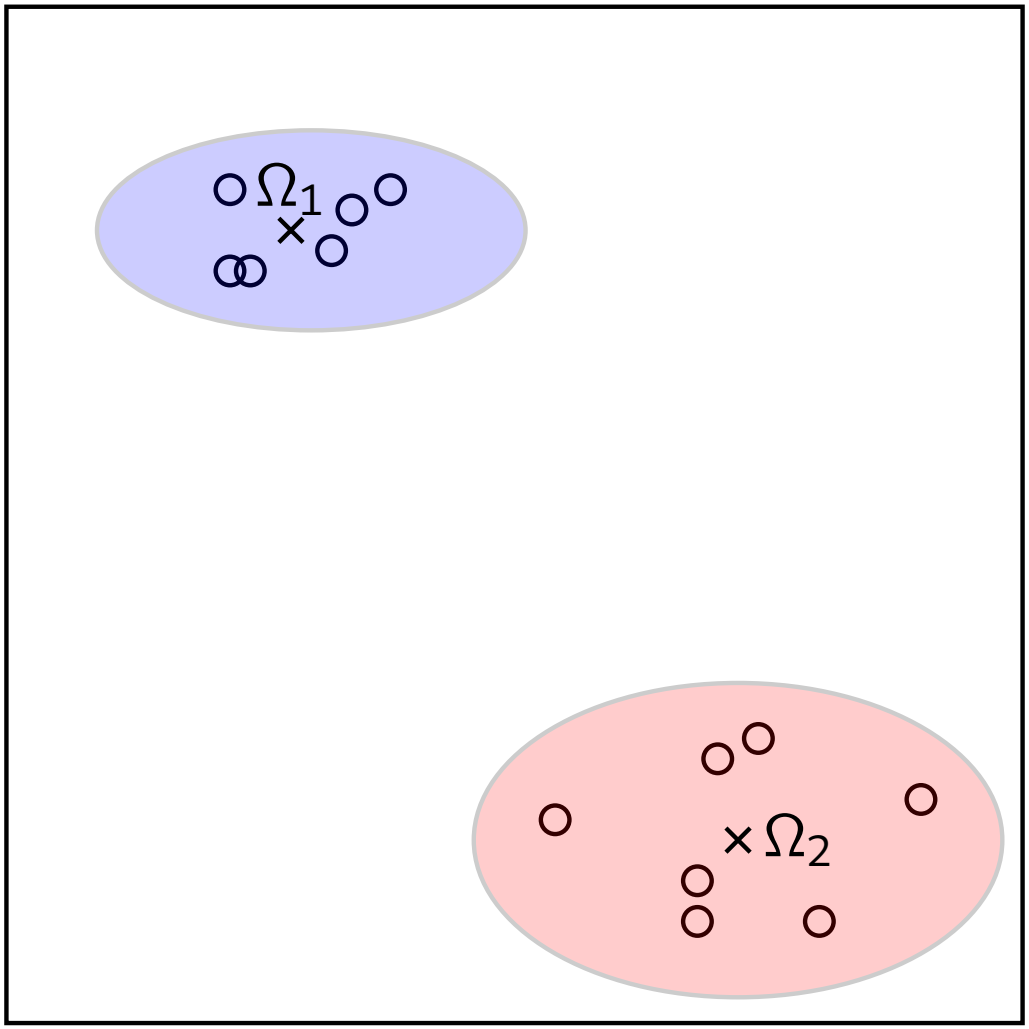
5 Conclusion

6 Lab session

# Clustering: the intuition



# Clustering: the intuition



# Clustering with centroids

A common way to perform clustering is to rely on distances to *centroids* (i.e. the centers of the clusters).

## Principle

- Choose $K$ cluster centroids $\Omega_1, \ldots, \Omega_K \in \mathbb{R}^d$,
- Each vector $\mathbf{x}$ belongs to the cluster of its closest centroid (denoted $\Omega_{q(\mathbf{x})}$),
- Error (*inertia*): sum of the squared distances between the points and their closest centroid

$$E \triangleq \sum_{\mathbf{x} \in X} \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2^2$$

**For a fixed $K$: the smaller the error, the better the clustering.**

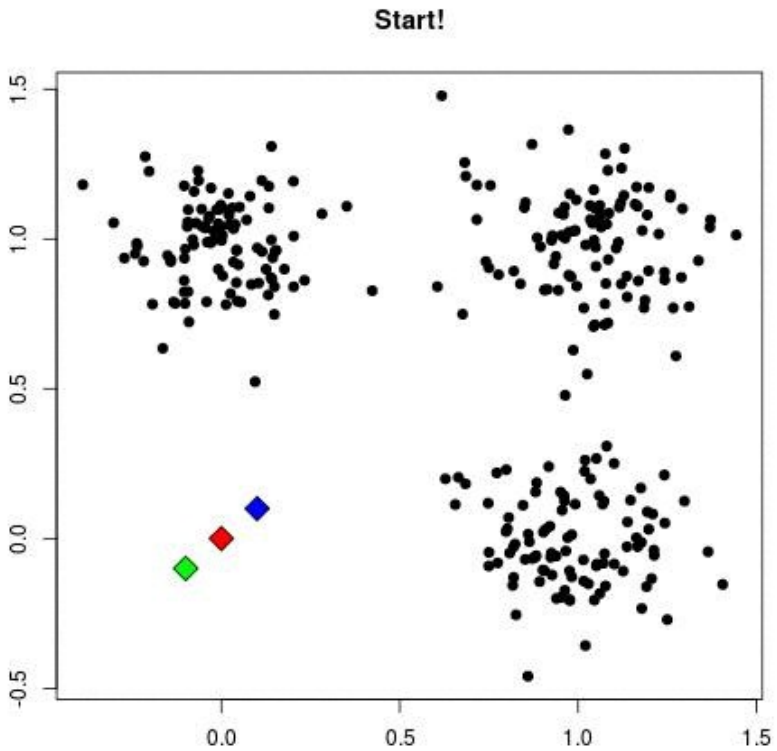
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

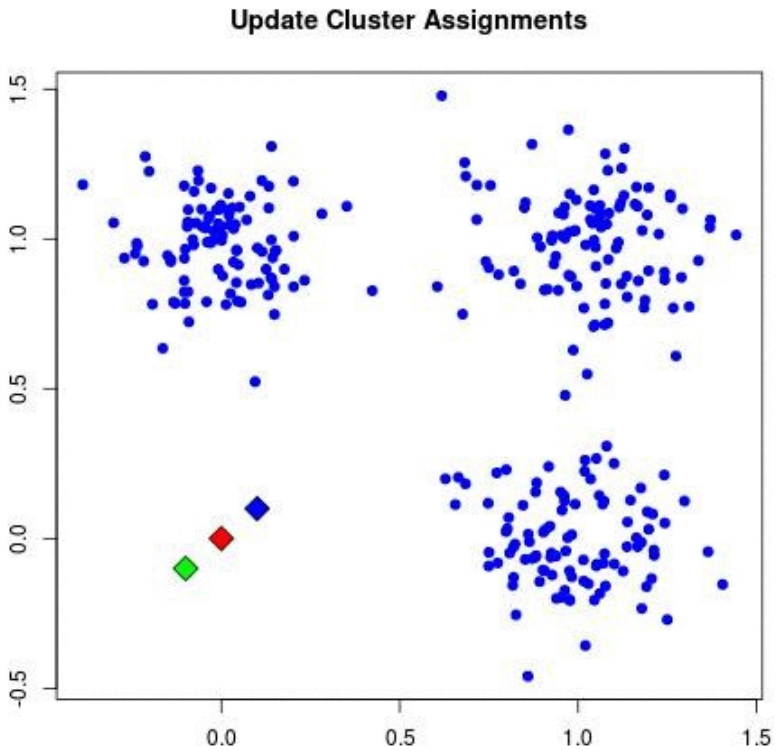
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

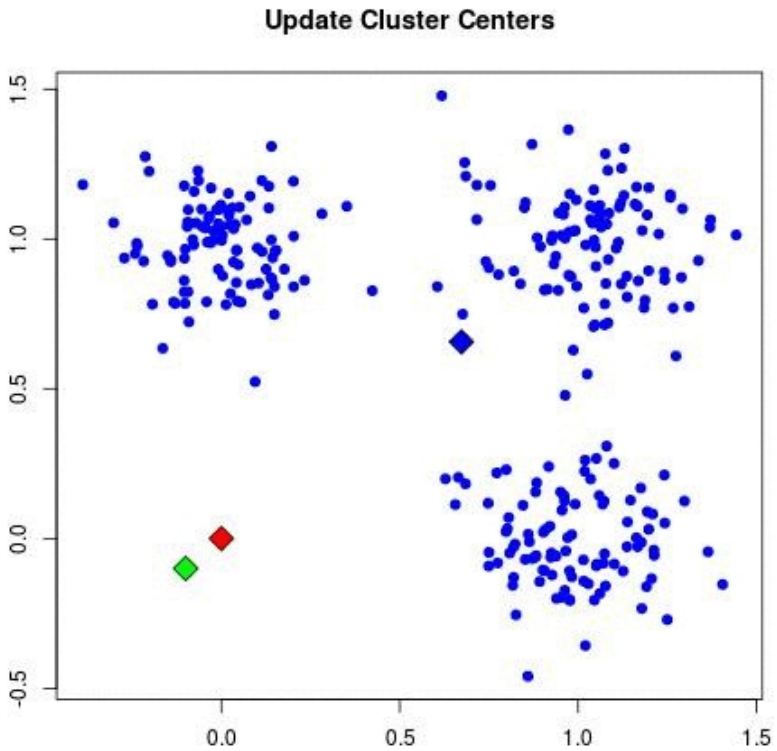
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

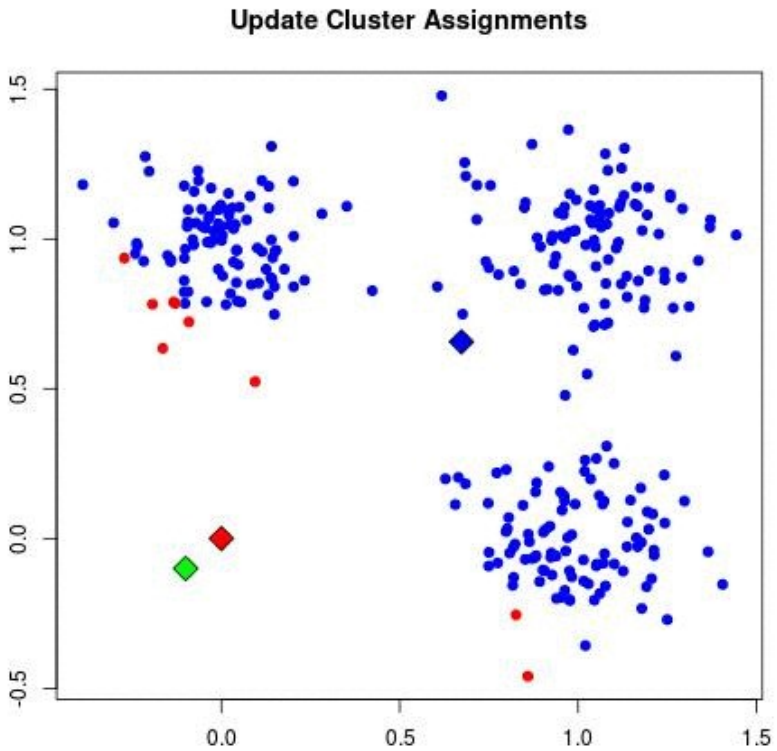
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

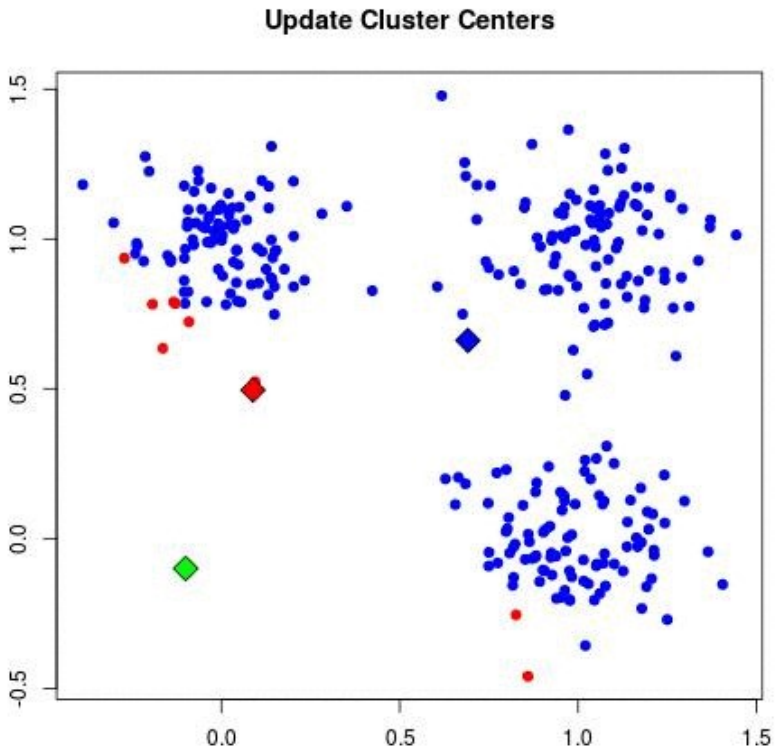
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

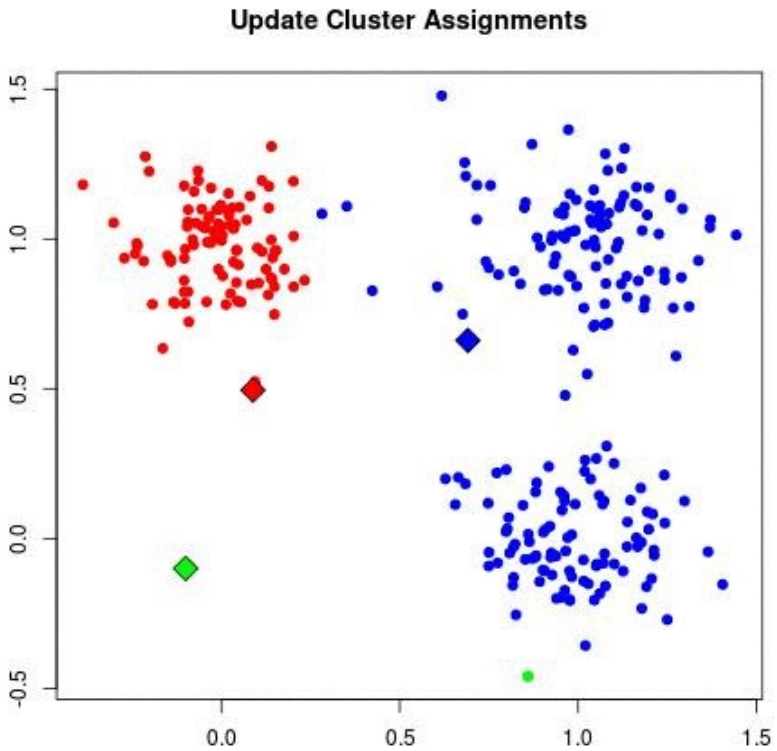
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

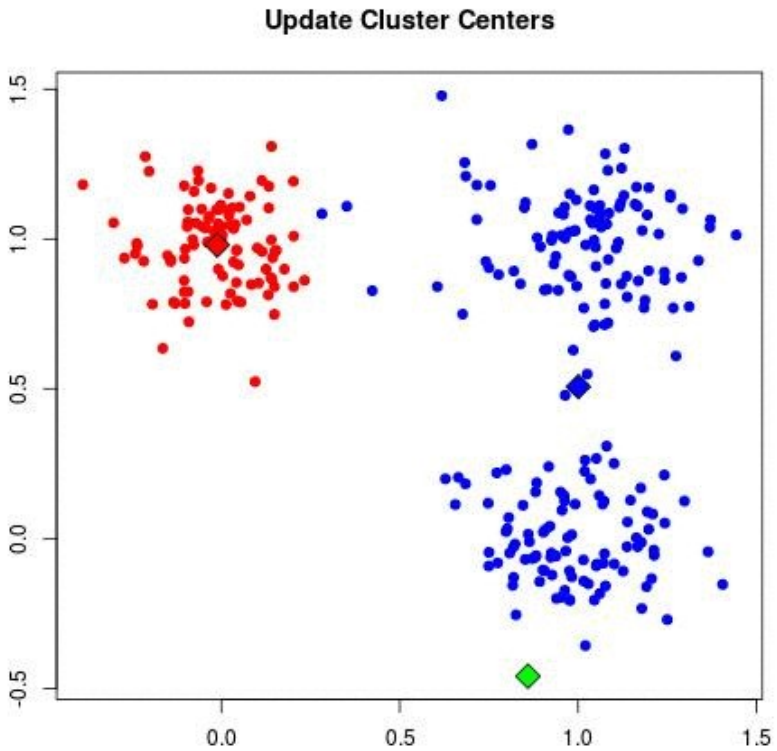
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

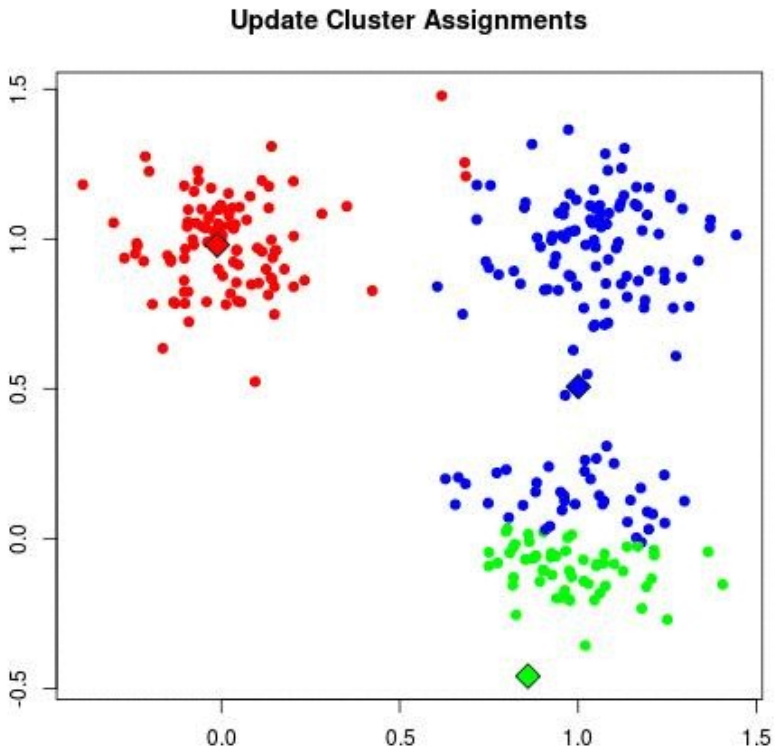
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

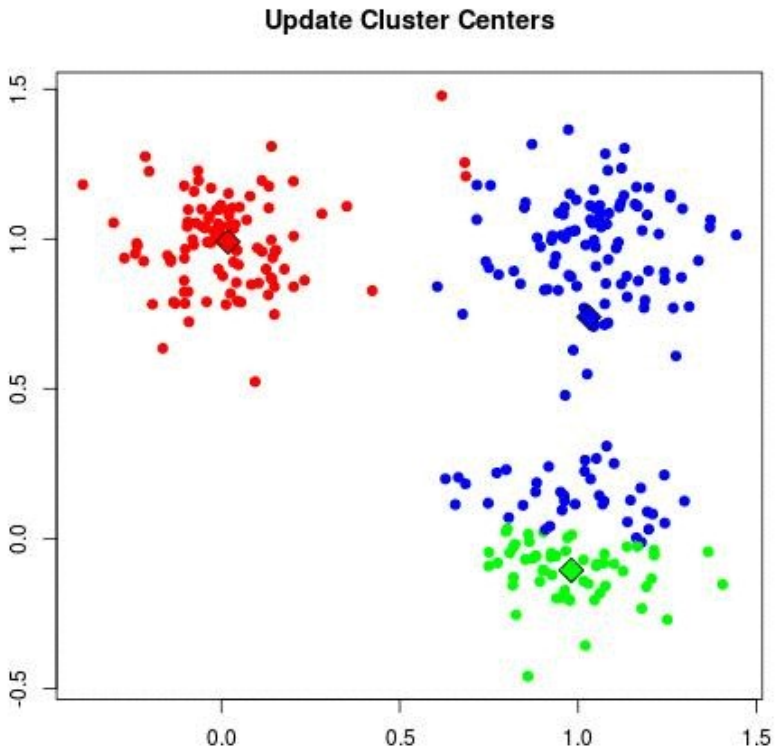
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

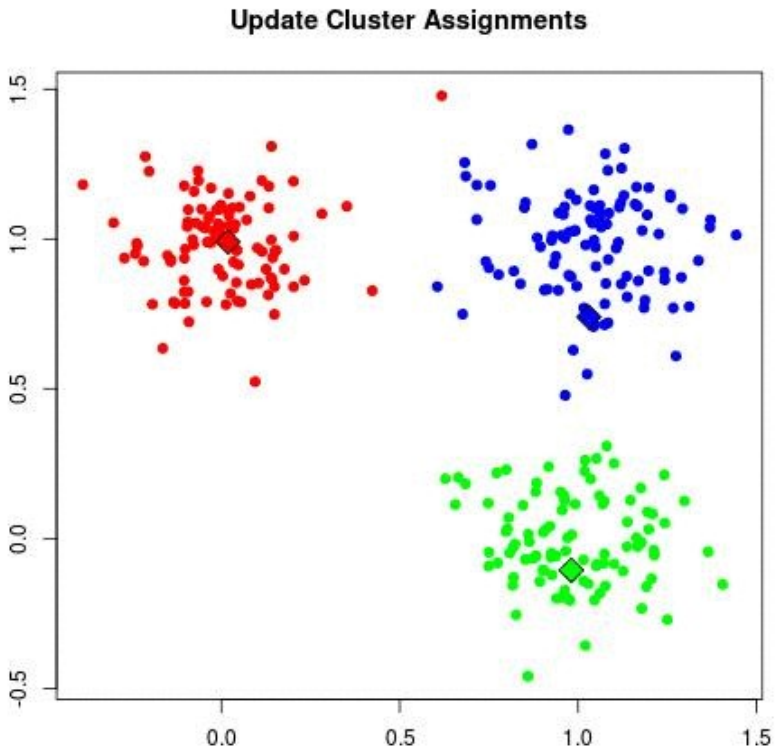
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

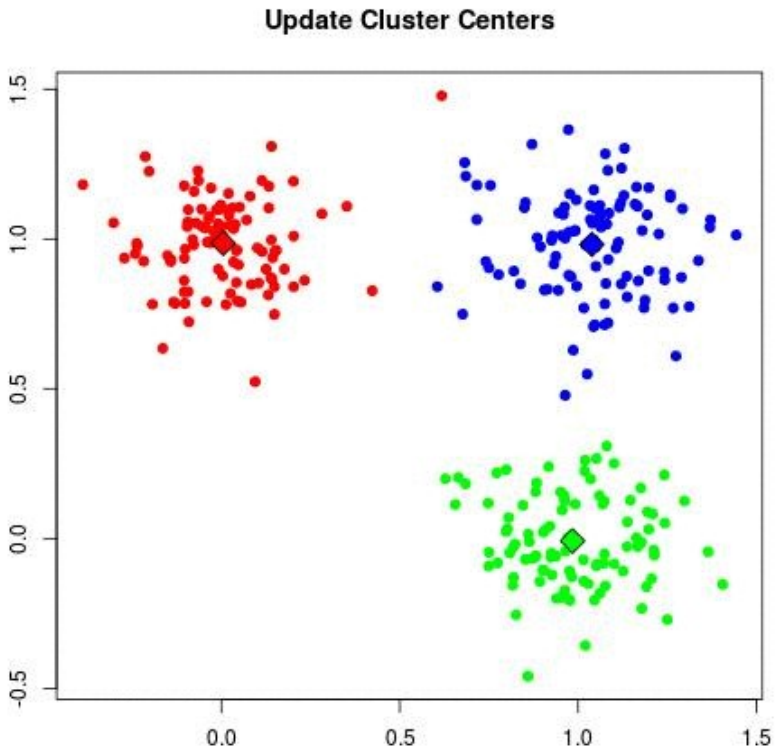
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat .

Converges, but to a **local** solution (which depends on the initialization).




Reference: https://mubaris.com/posts/kmeans-clustering/

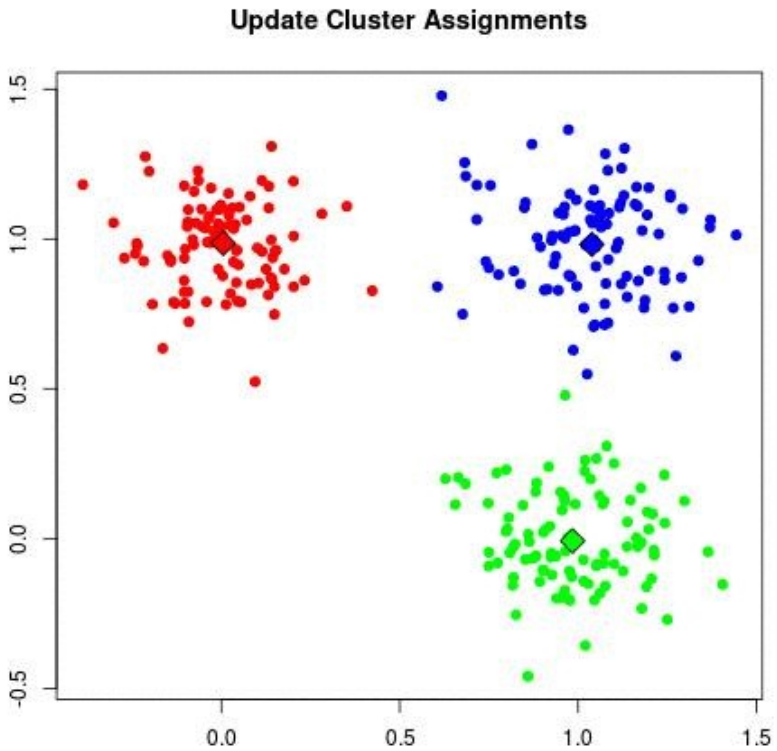
# A standard algorithm: K-Means

## K-means algorithm

First: initialize $K$ cluster centroids.

1. Assign each data point to the cluster of closest centroid.
2. Compute the new centroids as the average of the data points in each cluster.
3. Repeat until convergence.

Converges, but to a **local** solution (which depends on the initialization).



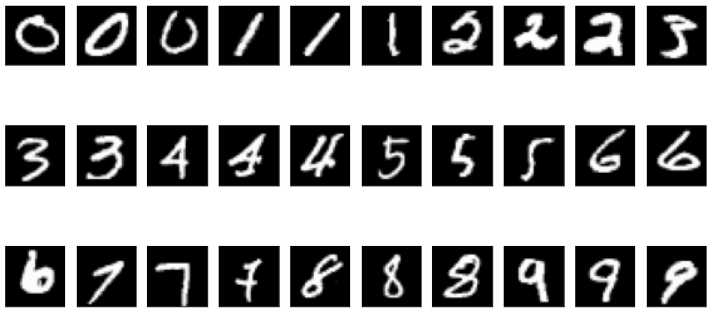

Reference: https://mubaris.com/posts/kmeans-clustering/

# In practice: K-Means on MNIST

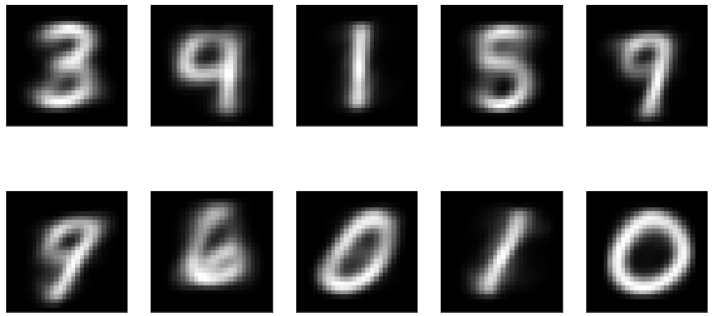
## Clustering MNIST

Using $K$-means algorithm with $K = 10$.
$\rightarrow$ Clusters capture, to some extent, the different digits.

Original MNIST examples

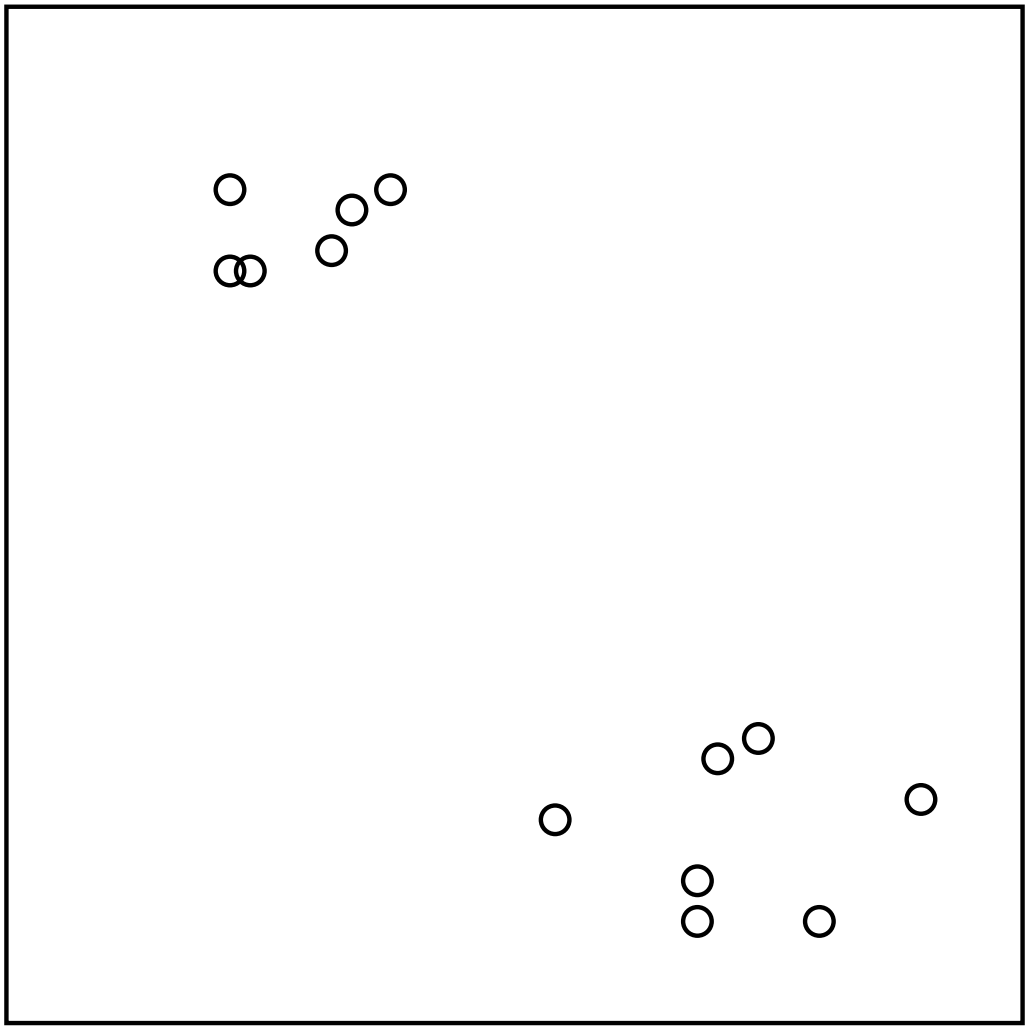


Cluster centroids

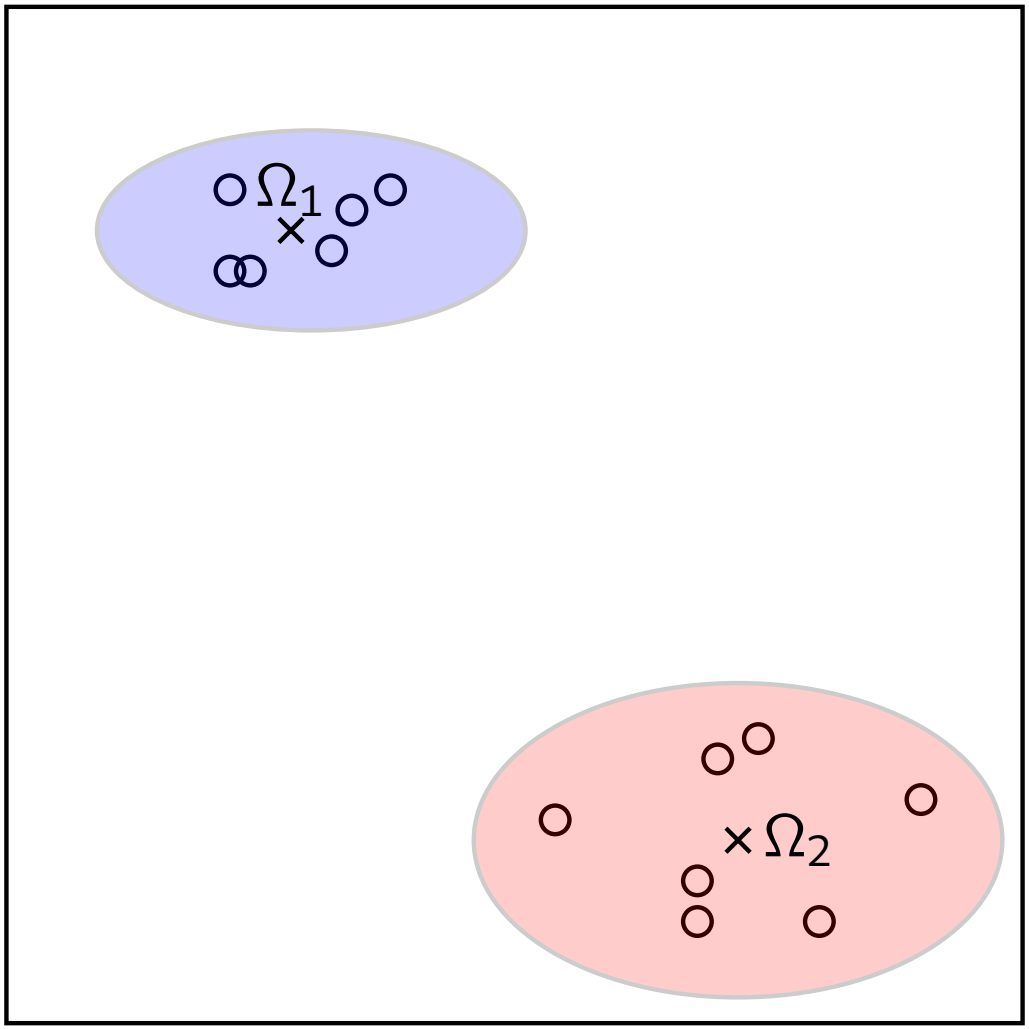


Note: we recall that images are vectorized for the clustering to make sense! They are only displayed in 2D to be interpretable.

# Impact of the choice of $K$

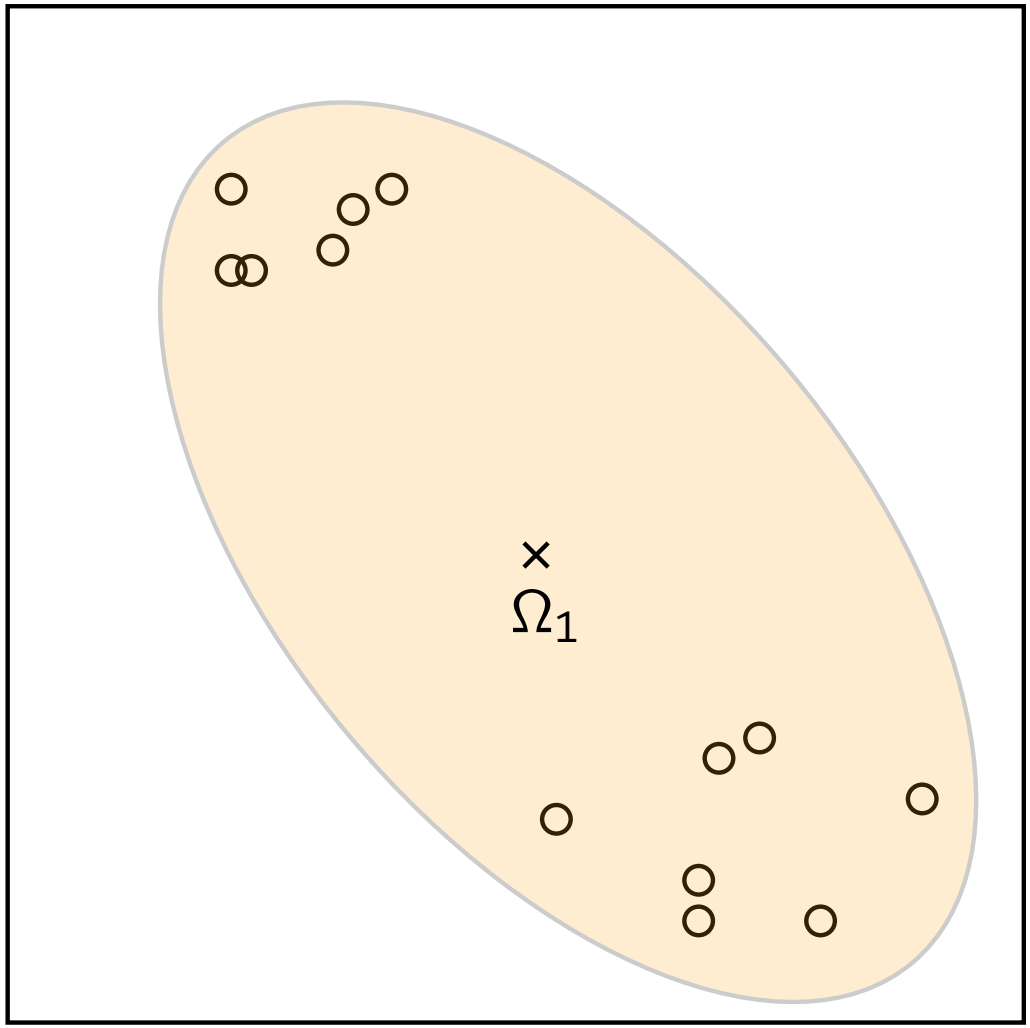


# Impact of the choice of $K$



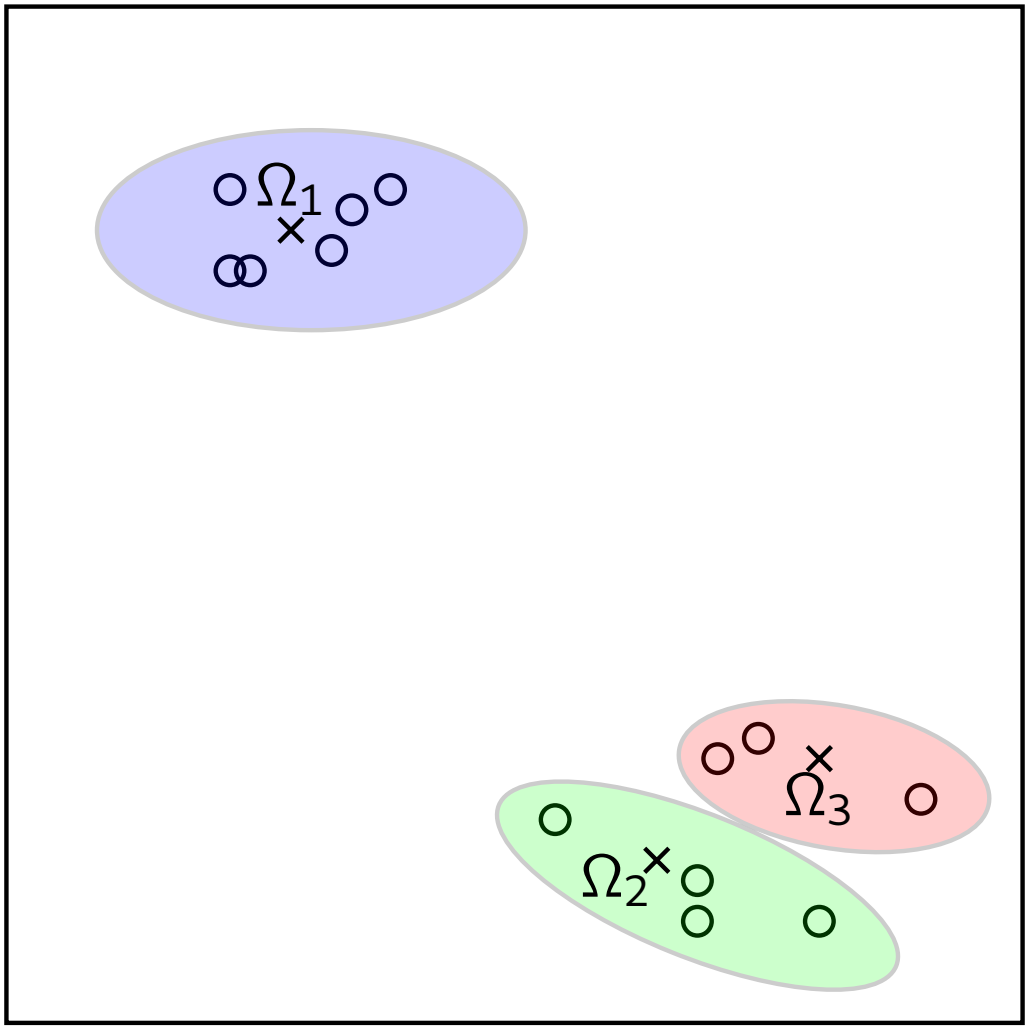

$K = 2$

# Impact of the choice of $K$



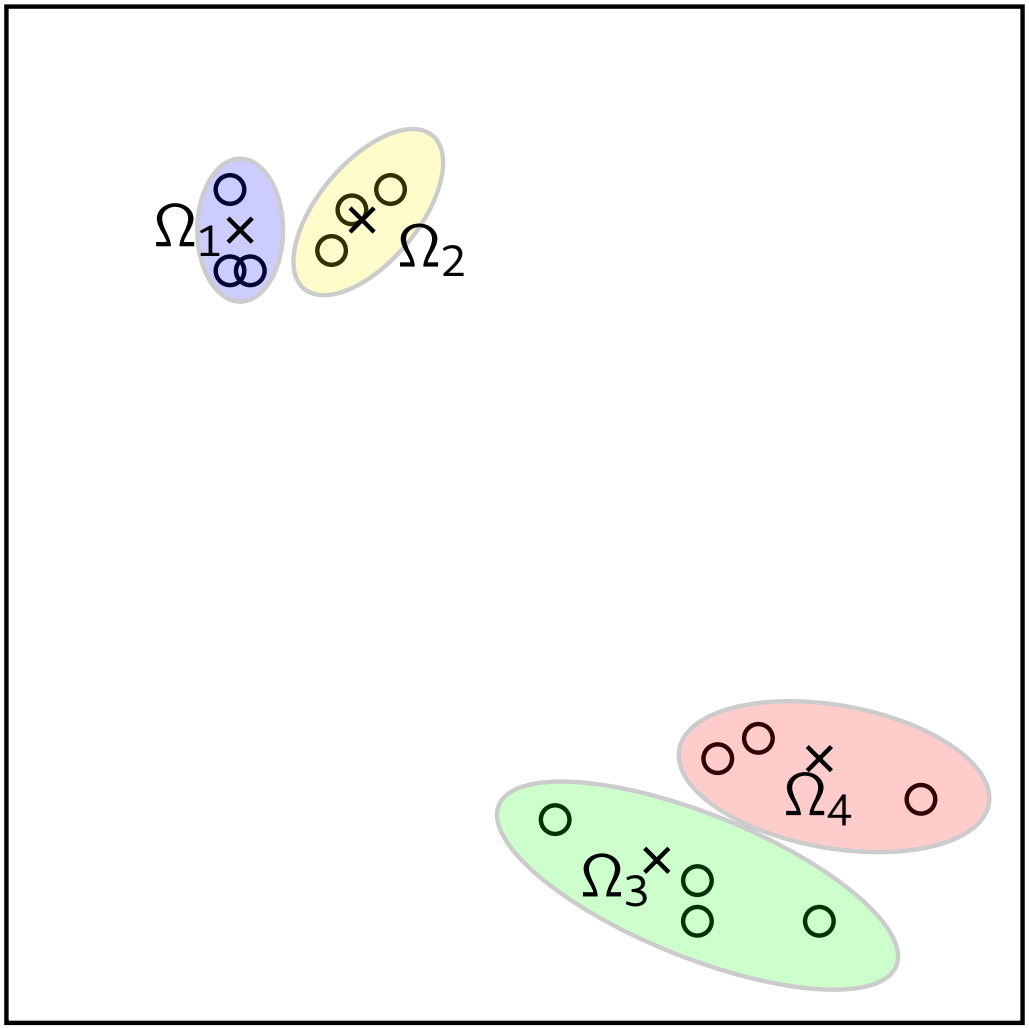

$K = 1$

# Impact of the choice of $K$



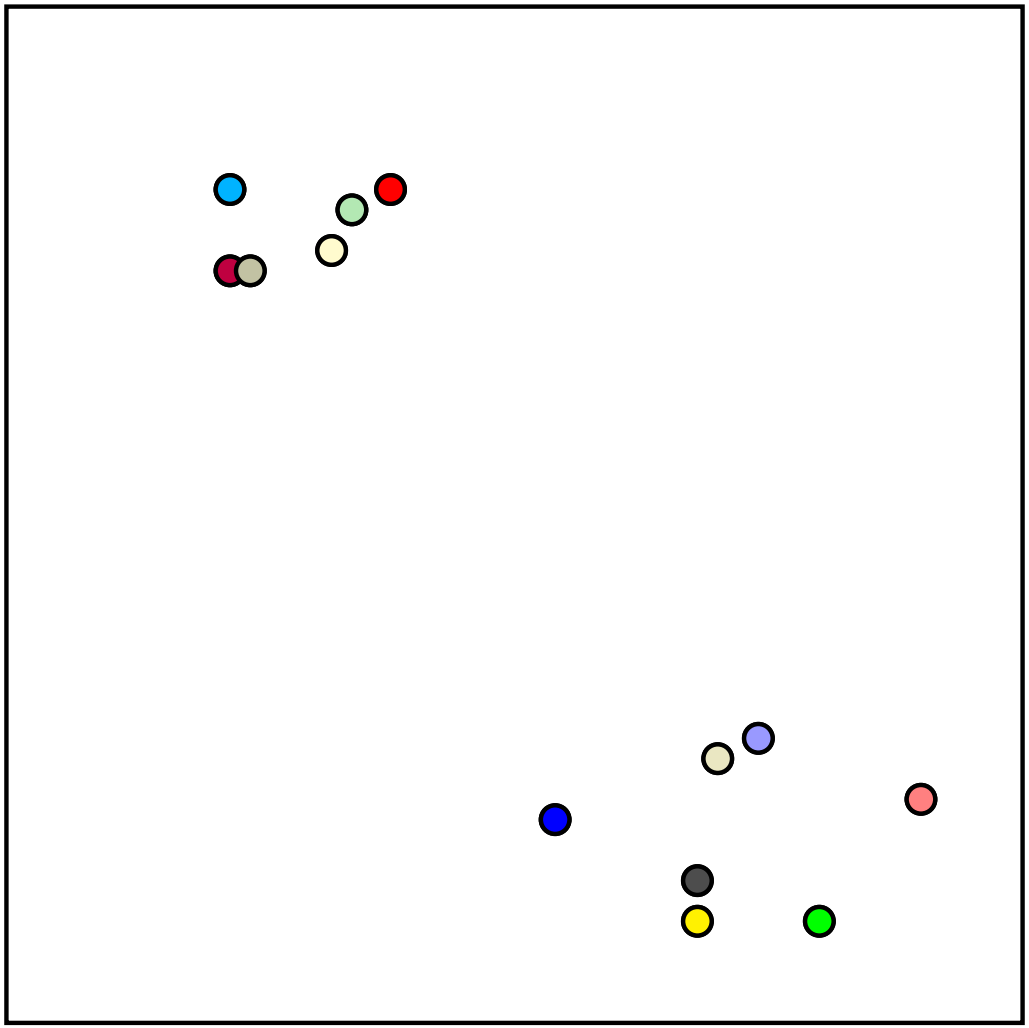

$K = 3$

# Impact of the choice of $K$




$K = 4$

# Impact of the choice of $K$



$K = N$

(each data point is its own centroid)

# The error alone cannot choose $K$

## Optimal error

- $E_{opt_K}$: the smallest possible error with $K$ clusters,
- Increasing $K$ can only decrease it.




$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

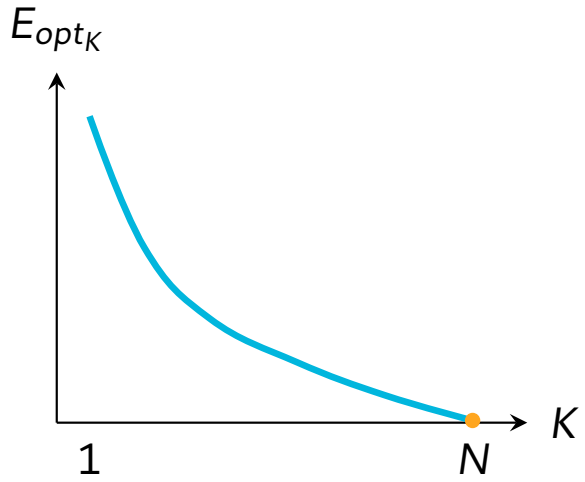
**Minimizing the error always leads to $K = N$...where nothing is learned!**

Never monitor the error alone to choose $K$: it is a trade-off between **error** and **compression** (more in the evaluation section).

# The error alone cannot choose $K$

## Optimal error

- $E_{opt_K}$: the smallest possible error with $K$ clusters,
- Increasing $K$ can only decrease it.




$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

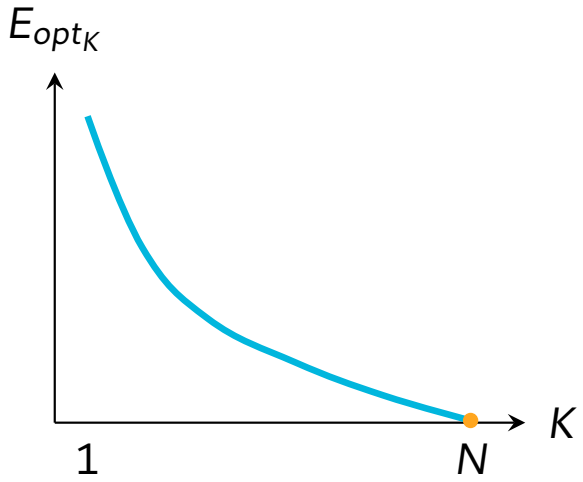
**Minimizing the error always leads to $K = N$...where nothing is learned!**

Never monitor the error alone to choose $K$: it is a trade-off between **error** and **compression** (more in the evaluation section).

# The error alone cannot choose $K$

## Optimal error

- $E_{opt_K}$: the smallest possible error with $K$ clusters,
- Increasing $K$ can only decrease it.




$$0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \cdots \leq E_{opt_2} \leq E_{opt_1} = N \cdot var(X)$$

**Minimizing the error always leads to $K = N$...where nothing is learned!**

Never monitor the error alone to choose $K$: it is a trade-off between **error** and **compression** (more in the evaluation section).

# Plan


1 Clustering

2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

# Why reduce the dimension?

High dimensional data is hard to handle (to visualize, to process, curse of dimensionality):

- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

*How to visualize all your images at once? And does it makes sense to compare images when each dimension is a pixel?*

## Dimensionality reduction

- Goal: describe each point with **fewer coordinates** (2D/3D to visualize, a few dozen to preprocess),
- Hypothesis: real data does not fill its whole space, it lives on a much **smaller** set.

**But what does "living on a smaller set" mean?**

# Why reduce the dimension?

High dimensional data is hard to handle (to visualize, to process, curse of dimensionality):

- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

*How to visualize all your images at once? And does it makes sense to compare images when each dimension is a pixel?*

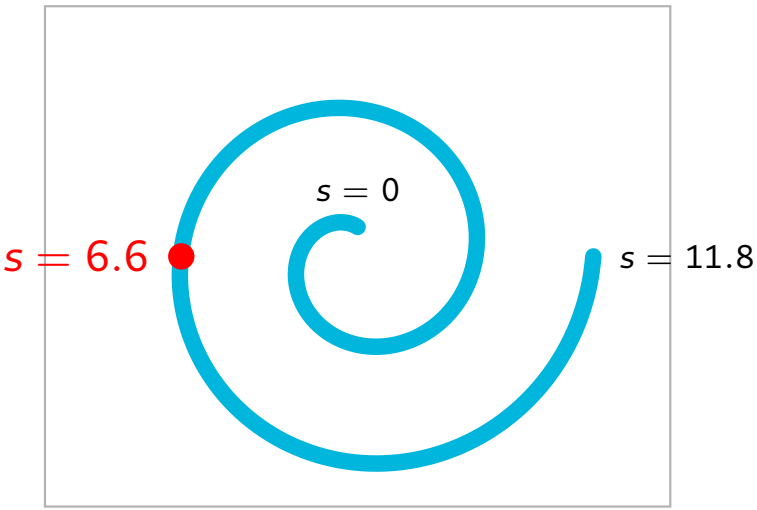
## Dimensionality reduction

- Goal: describe each point with **fewer coordinates** (2D/3D to visualize, a few dozen to preprocess),
- Hypothesis: real data does not fill its whole space, it lives on a much **smaller** set.

**But what does "living on a smaller set" mean?**

# Where does the data actually live?

## A rolled up sheet ("swiss roll")




Position in the plane: **2** numbers $(x, y)$

Position along the roll: **1** number $(s)$

## One digit, rotated




Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

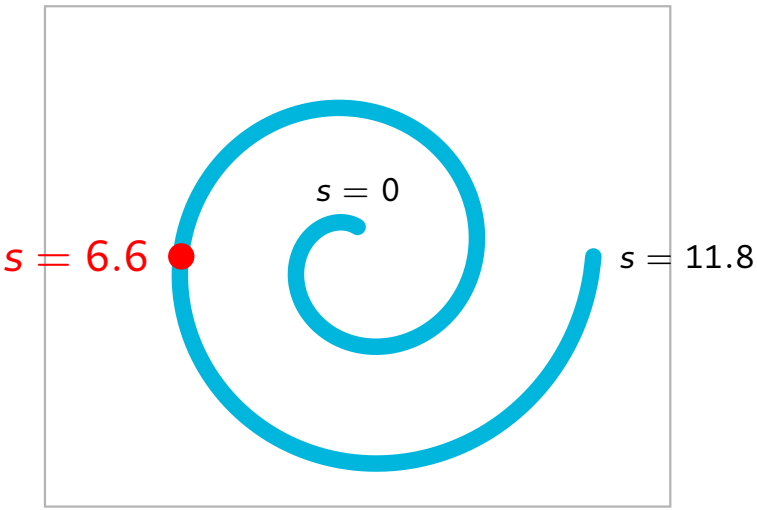
## Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

**Straight line distances can be misleading: follow the roll!**

# Where does the data actually live?

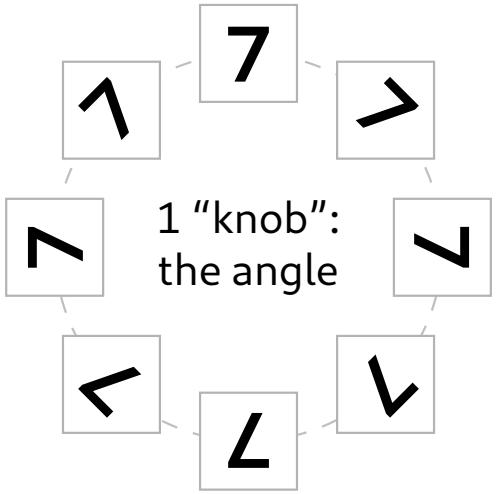
## A rolled up sheet ("swiss roll")




Position in the plane: **2** numbers $(x, y)$

Position along the roll: **1** number $(s)$

## One digit, rotated




Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

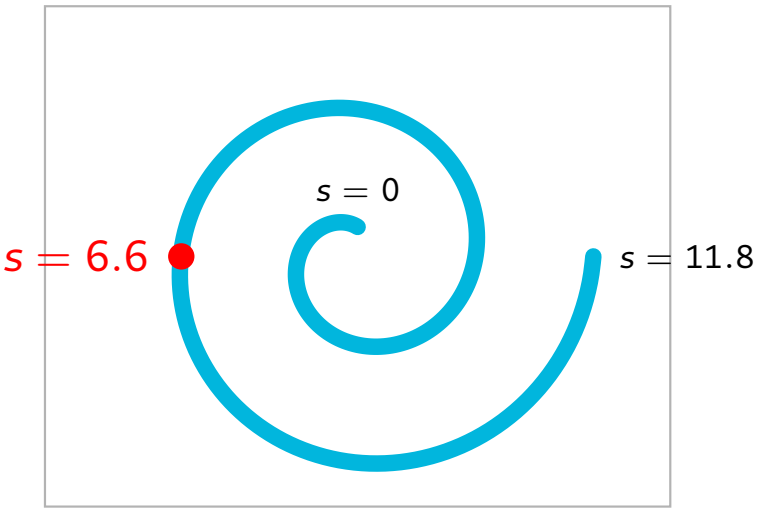
### Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

**Straight line distances can be misleading: follow the roll!**

# Where does the data actually live?

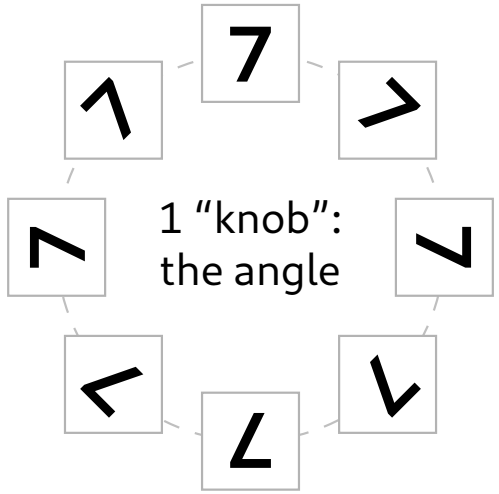
## A rolled up sheet ("swiss roll")




Position in the plane: **2** numbers $(x, y)$

Position along the roll: **1** number $(s)$

## One digit, rotated




Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

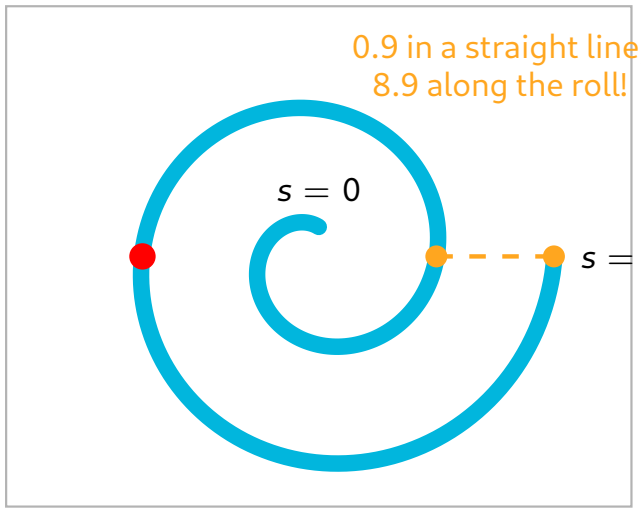
## Intrinsic dimension

The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

Straight line distances can be misleading: follow the roll!

# Where does the data actually live?

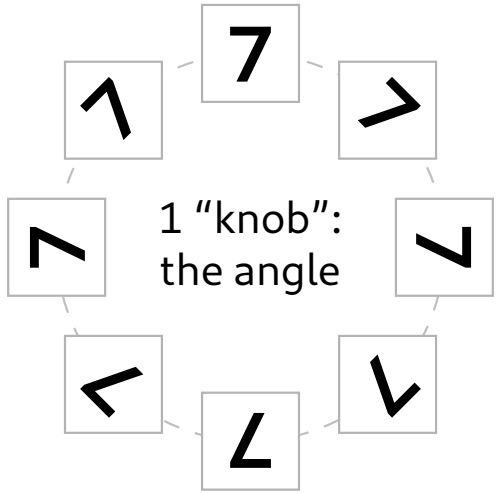
## A rolled up sheet ("swiss roll")




Position in the plane: **2** numbers $(x, y)$

Position along the roll: **1** number $(s)$

## One digit, rotated




Each image: **784** numbers (pixels)

Generated by: **1** number (the angle)

## Intrinsic dimension

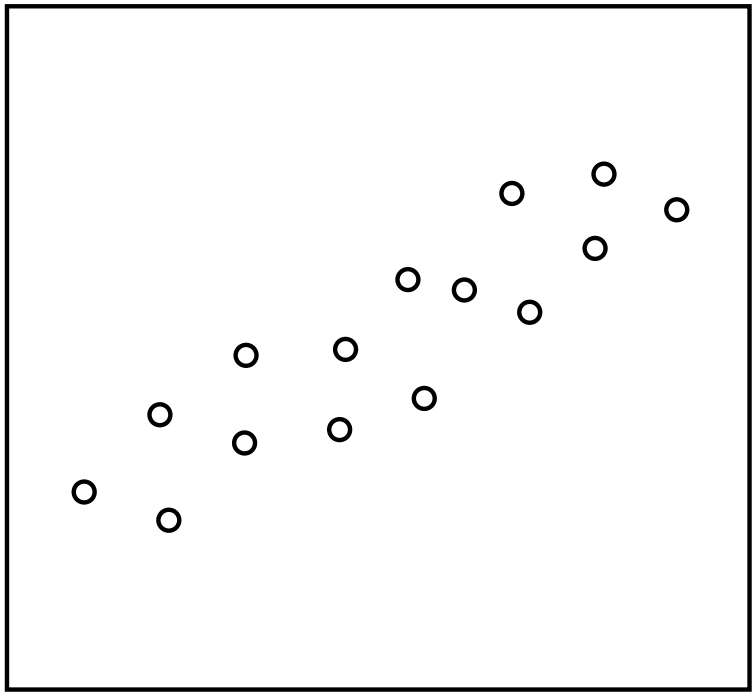
The number of dimensions needed to generate the data: the data is *stored* in $\mathbb{R}^d$, but *lives* on a much smaller set.

**Straight line distances can be misleading: follow the roll!**

# Principal Component Analysis (PCA): how it works

## Principle

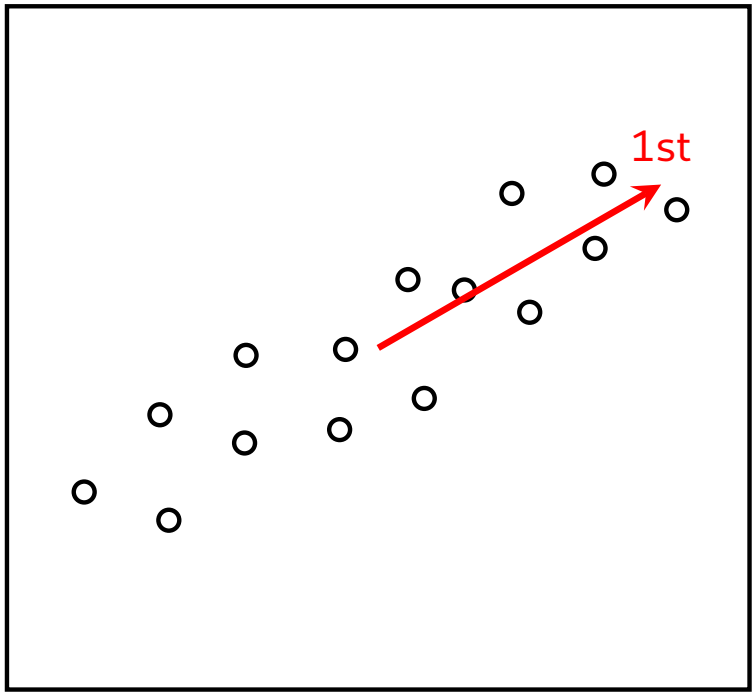
- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first $K$ components, and **project** the points on them: $K$ coordinates instead of $d$.



# Principal Component Analysis (PCA): how it works

## Principle

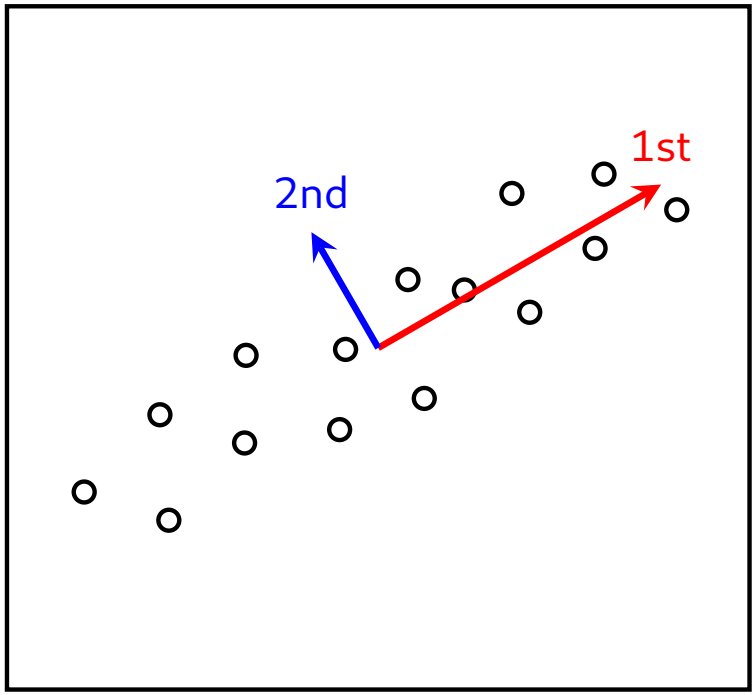
- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first $K$ components, and **project** the points on them: $K$ coordinates instead of $d$.



# Principal Component Analysis (PCA): how it works

## Principle

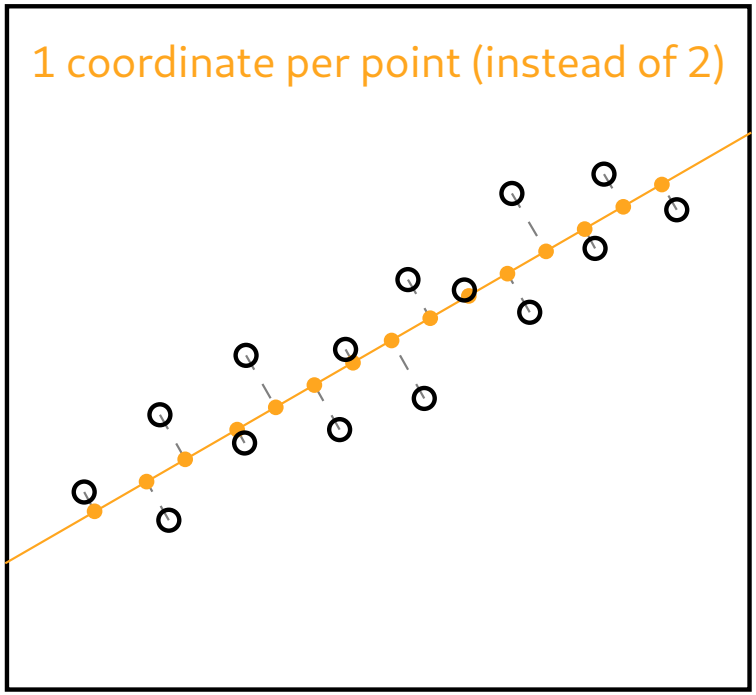
- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first $K$ components, and **project** the points on them: $K$ coordinates instead of $d$.



# Principal Component Analysis (PCA): how it works

## Principle

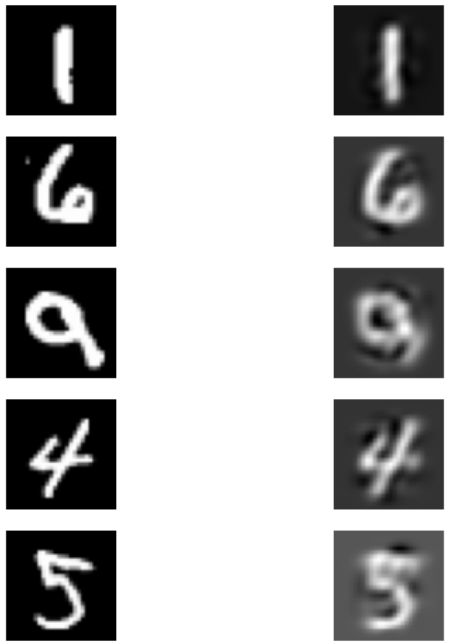
- Find the direction along which the data **varies the most** (1st component),
- Then the next one, **orthogonal** to the previous ones, and so on: components are **ordered by importance** (variance explained),
- Keep only the first $K$ components, and **project** the points on them: $K$ coordinates instead of $d$.



# PCA in practice

## What you need to know

- **Linear**: each new coordinate is a weighted sum of the original features,
- Keeps as much **variance** as possible with $K$ components,
- Once fitted, it can project **new points**,
- Uses: **visualization** ($K = 2$), **preprocessing** before another model (fewer features, less noise), compression.

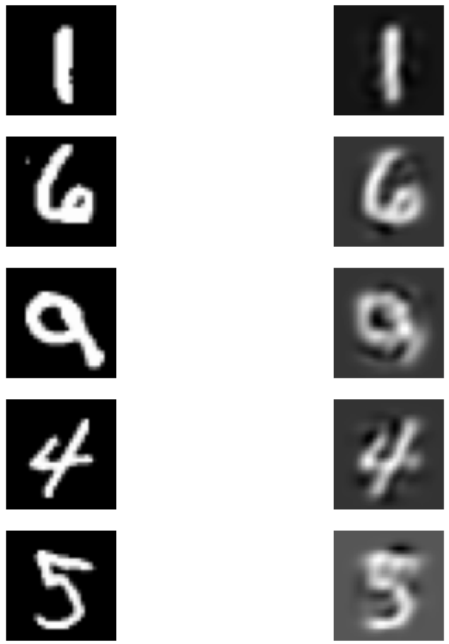


MNIST digits rebuilt from only $K = 32$ coordinates (instead of $d = 784$ pixels).

# PCA in practice

## What you need to know

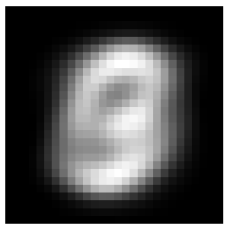
- **Linear**: each new coordinate is a weighted sum of the original features,
- Keeps as much **variance** as possible with $K$ components,
- Once fitted, it can project **new points**,
- Uses: **visualization** ($K = 2$), **preprocessing** before another model (fewer features, less noise), compression.



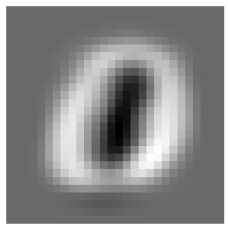
MNIST digits rebuilt from only $K = 32$ coordinates (instead of $d = 784$ pixels).

# PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time

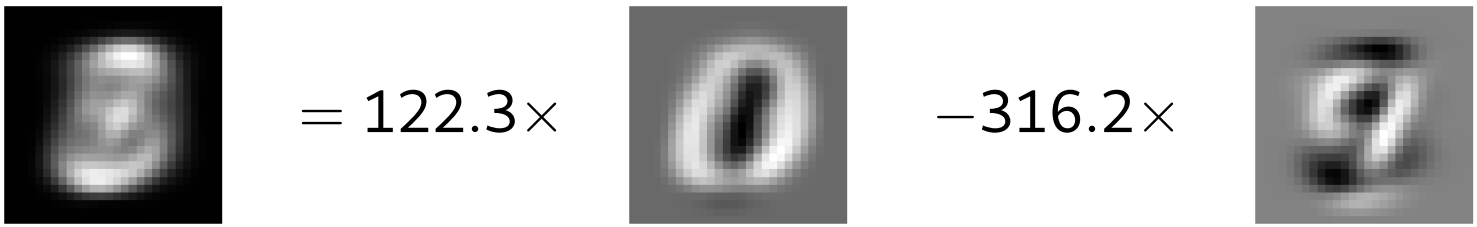


$= 122.3 \times$



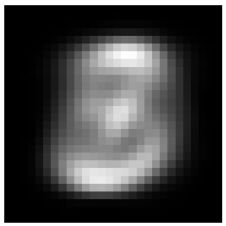
# PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time

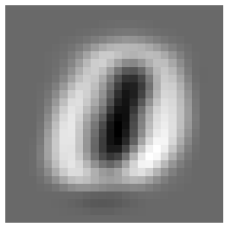


# PCA on MNIST: step by step

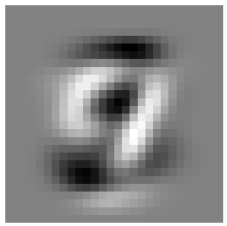
Rebuilding one digit, adding one component at a time



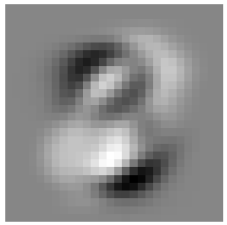
$= 122.3\times$



$-316.2\times$

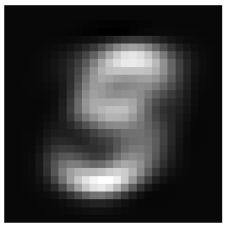
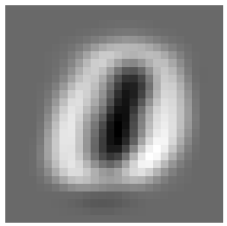
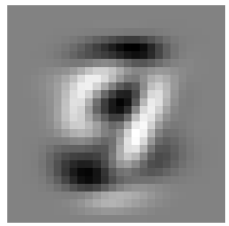


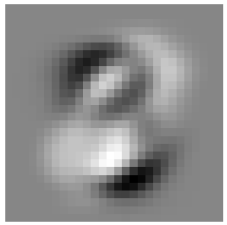
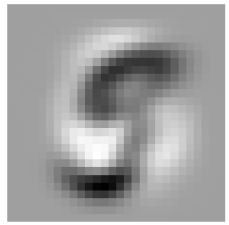
$-51.13\times$



# PCA on MNIST: step by step

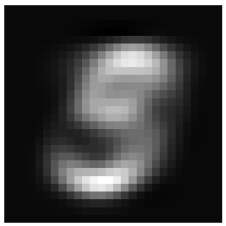
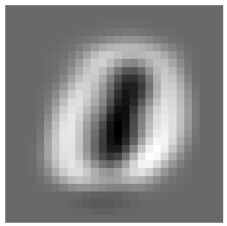
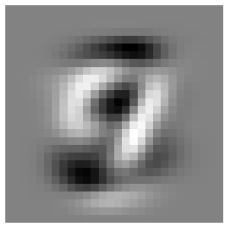
Rebuilding one digit, adding one component at a time

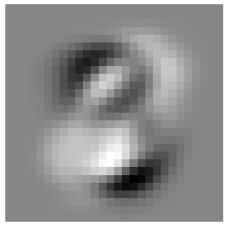
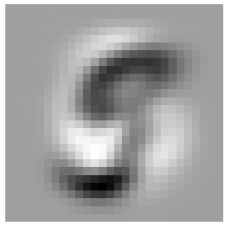

$= 122.3\times$

$-316.2\times$


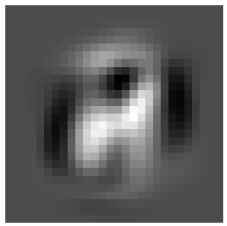
$-51.13\times$

$-556.9\times$


# PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time

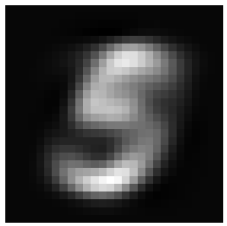
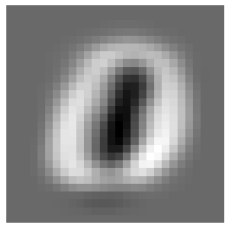
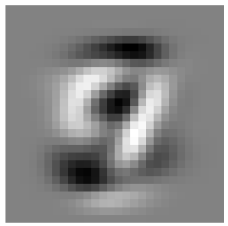

$= 122.3\times$

$-316.2\times$


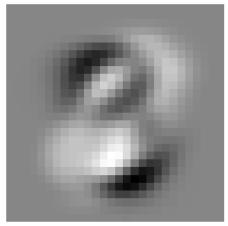
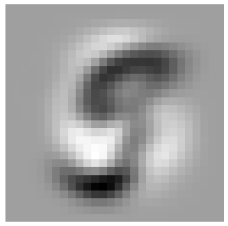
$-51.13\times$

$-556.9\times$


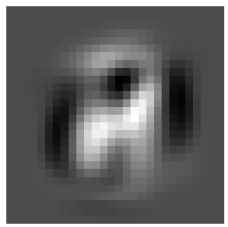
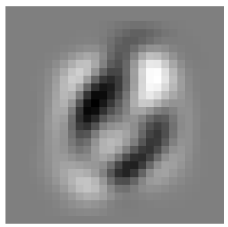
$-49.2\times$


# PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time


$= 122.3\times$

$-316.2\times$


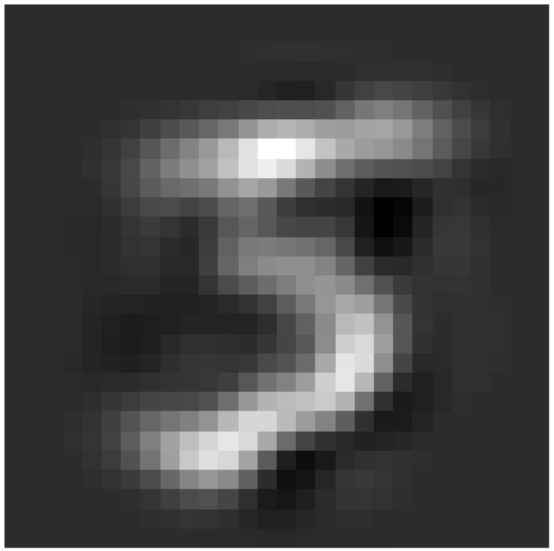
$-51.13\times$

$-556.9\times$


$-49.2\times$

$-217.1\times$

$\ldots$

# PCA on MNIST: step by step

Rebuilding one digit, adding one component at a time

Reconstruction with all 32 components:



# t-SNE: a nonlinear method for visualization

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data. Unlike PCA, it is not restricted to straight directions.

## How it works (simplified)

1. In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
2. In 2D: define the same kind of distribution for the projected points,
3. Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

## Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

# t-SNE: a nonlinear method for visualization

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data. Unlike PCA, it is not restricted to straight directions.

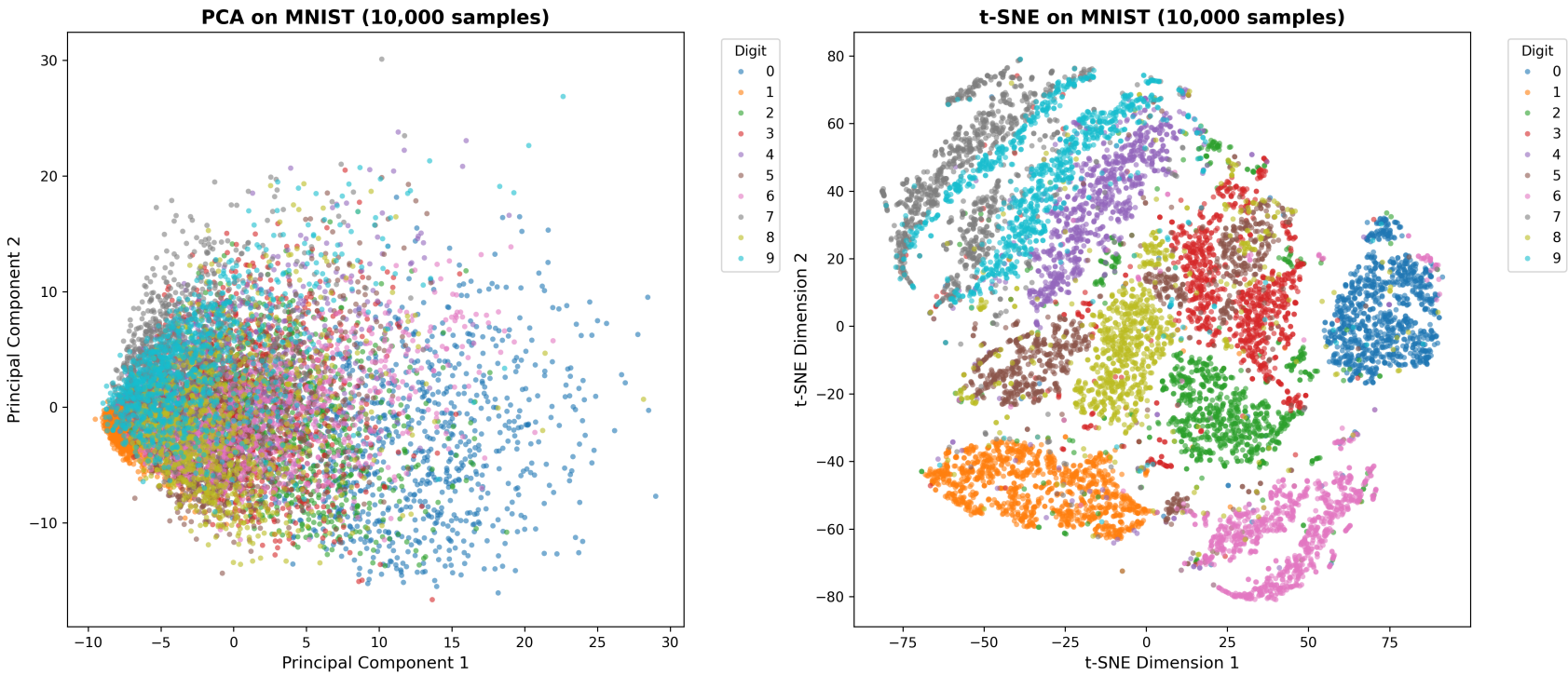
## How it works (simplified)

1. In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
2. In 2D: define the same kind of distribution for the projected points,
3. Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

## Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

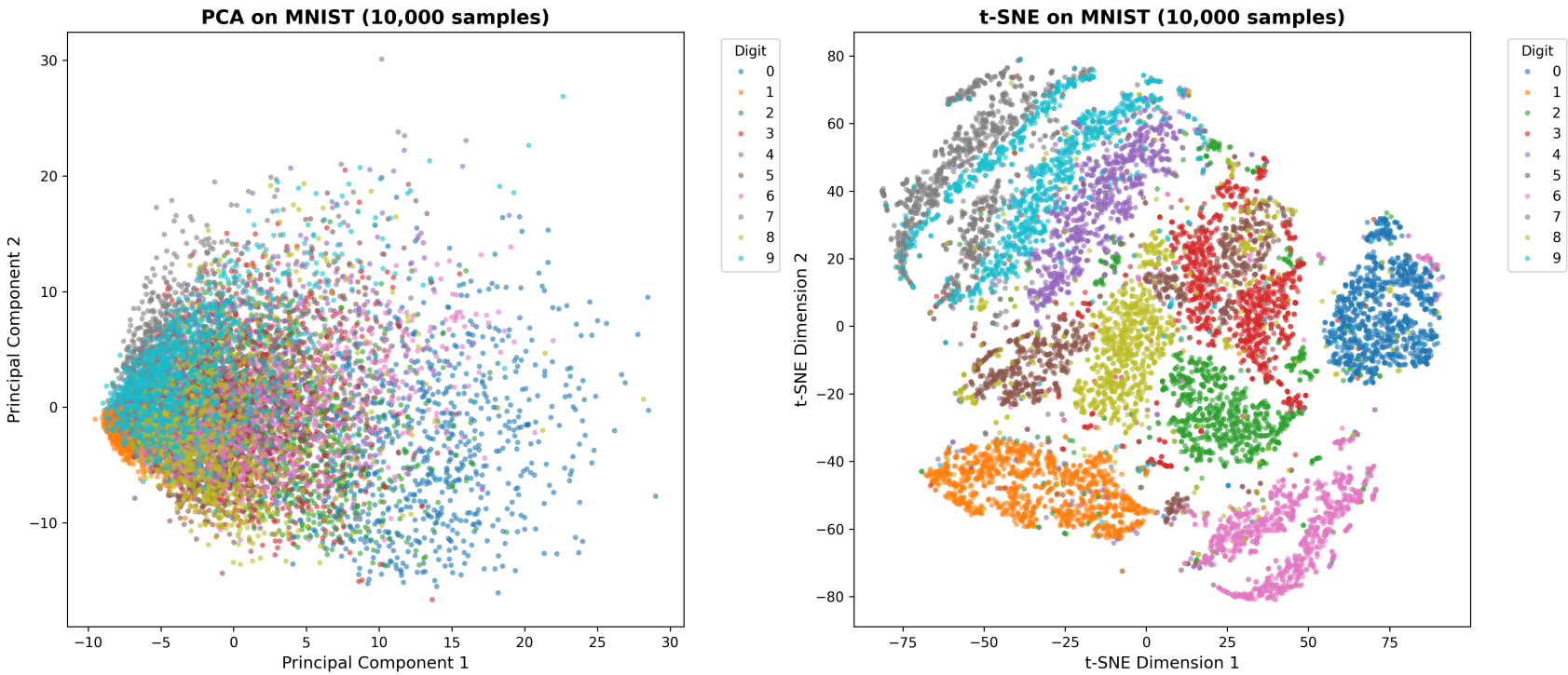
# PCA vs t-SNE




Colors = digit classes (**not used** by the algorithms, only for display).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

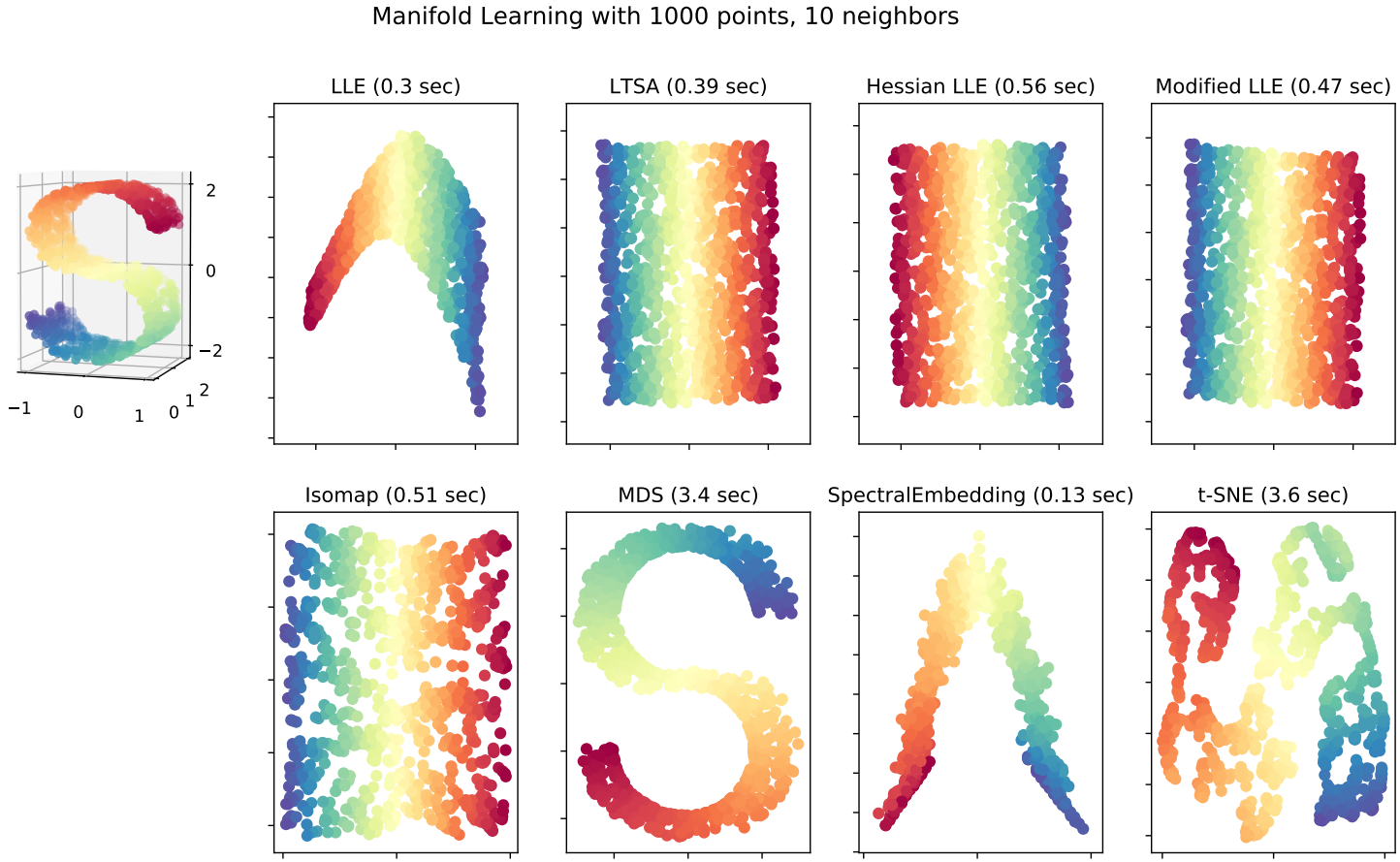
# PCA vs t-SNE




Colors = digit classes (**not used** by the algorithms, only for display).

**Where PCA preserves global variance, t-SNE preserves local neighborhoods.**

# Going further: manifold learning, a family of methods




t-SNE is one member of a larger family of nonlinear methods ("manifold learning"), each preserving a different aspect of the data.

Source : Manifold module, sklearn website

# Plan


1 Clustering

2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use $X$ and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use $X$ and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use $X$ and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use $X$ and the result.
  "Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.
  "Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

# Fully unsupervised clustering metrics

## Inertia

The error $E$ of slide 10: sum of squared distances to the centroids.
Never increases with $K$ (0 for $K = N$) $\Rightarrow$ never use it alone.

## Silhouette score

For each sample: $s = \dfrac{b - a}{\max(a, b)} \in [-1, 1]$

- $a$: mean distance to the points of **its own** cluster,
- $b$: mean distance to the points of the **nearest other** cluster.

$s \approx 1$: well placed; $s \approx 0$: on the border;
$s < 0$: probably in the wrong cluster.




here $b \gg a$:
$s$ close to 1

# Fully unsupervised clustering metrics

## Inertia

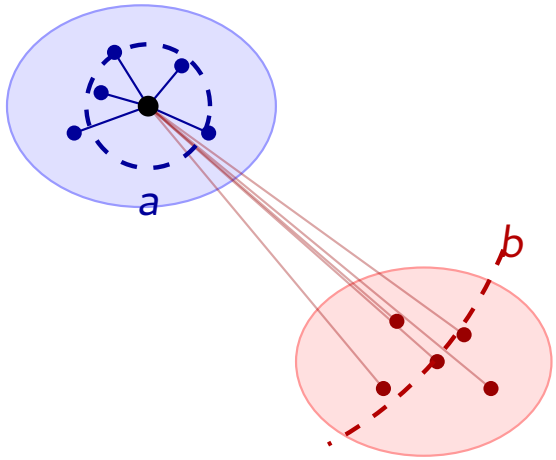
The error $E$ of slide 10: sum of squared distances to the centroids.
Never increases with $K$ ($0$ for $K = N$) $\Rightarrow$ never use it alone.

## Silhouette score

For each sample: $s = \dfrac{b - a}{\max(a, b)} \in [-1, 1]$

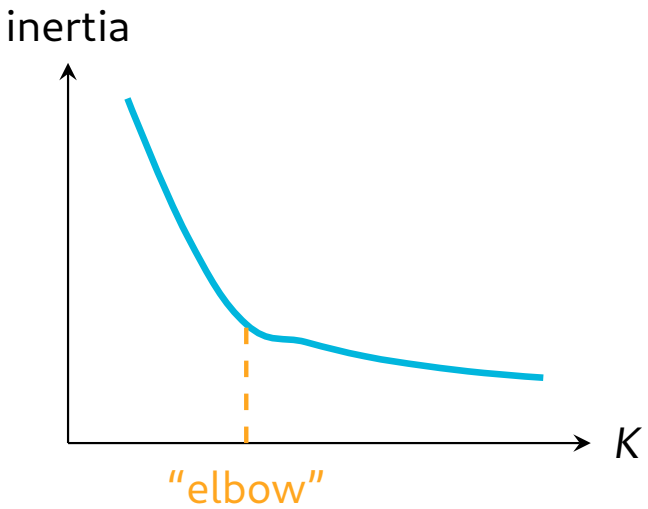
- $a$: mean distance to the points of **its own** cluster,
- $b$: mean distance to the points of the **nearest other** cluster.

$s \approx 1$: well placed; $s \approx 0$: on the border;
$s < 0$: probably in the wrong cluster.

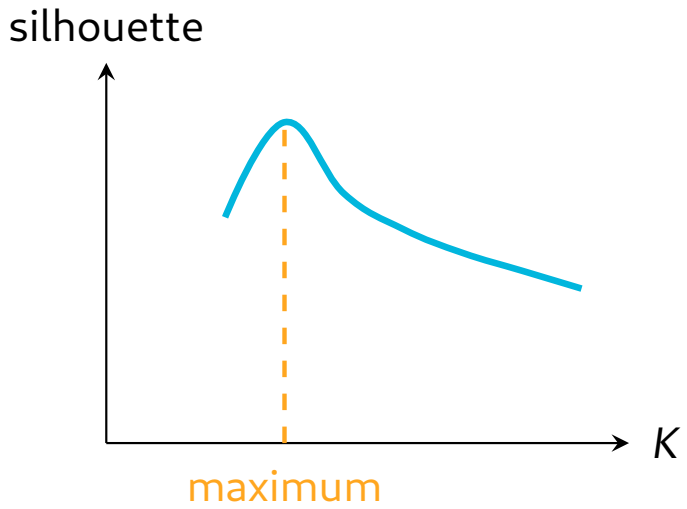



here $b \gg a$:
$s$ close to 1

# Back to choosing $K$: two heuristics




Elbow: look for the bend




Silhouette: look for the maximum

## In practice

- Both are heuristics: they may disagree, and real curves are noisy!
- Additionally, consider the stability across runs/subsamples, and whether the clusters make sense,
- $K$ may be imposed by the application (e.g. 5 customer profiles).

# External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).



homogeneous, not complete



complete, not homogeneous

**Warning: a low score may mean the clustering found *another*, valid structure.**

# External metrics: when labels do exist

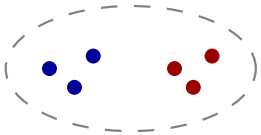
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).



homogeneous,
not complete



complete,
not homogeneous

**Warning: a low score may mean the clustering found *another*, valid structure.**

# External metrics: when labels do exist

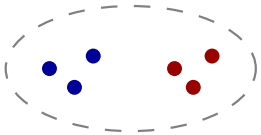
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index**: agreement between the clustering partitions and the labels. *Invariant to label permutations.*
- **Homogeneity**: each cluster contains only members of a single class,
- **Completeness**: all members of a class are in the same cluster,
- **V-measure**: harmonic mean of both (the "F1" of clustering).



homogeneous,
not complete



complete,
not homogeneous

**Warning: a low score may mean the clustering found *another*, valid structure.**

# Evaluating dimensionality reduction

## PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose $K$ ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only $K$ coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

## t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

**Ultimately: does it help the next step of your project?**

# Evaluating dimensionality reduction

## PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose $K$ ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only $K$ coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

## t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

**Ultimately: does it help the next step of your project?**

# Evaluating dimensionality reduction

## PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose $K$ ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only $K$ coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

## t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

**Ultimately: does it help the next step of your project?**

# Evaluating dimensionality reduction

## PCA

- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose $K$ ("keep 95%"),
- **Reconstruction error**: how much is lost when rebuilding the data from only $K$ coordinates,
- Careful: reconstruction is perfect for $K = d$, but nothing is reduced!

## t-SNE (and other nonlinear methods)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

**Ultimately: does it help the next step of your project?**

# Plan


1 Clustering

2 Dimensionality reduction

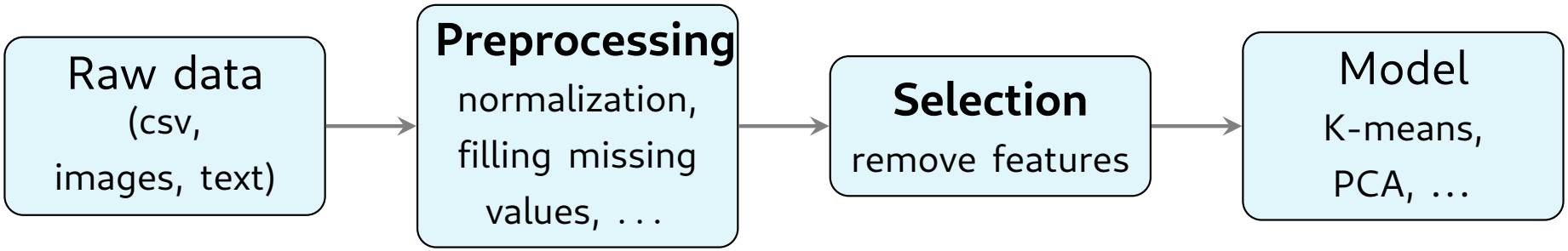
3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

# Features in standard Machine Learning pipelines




## Why it matters here

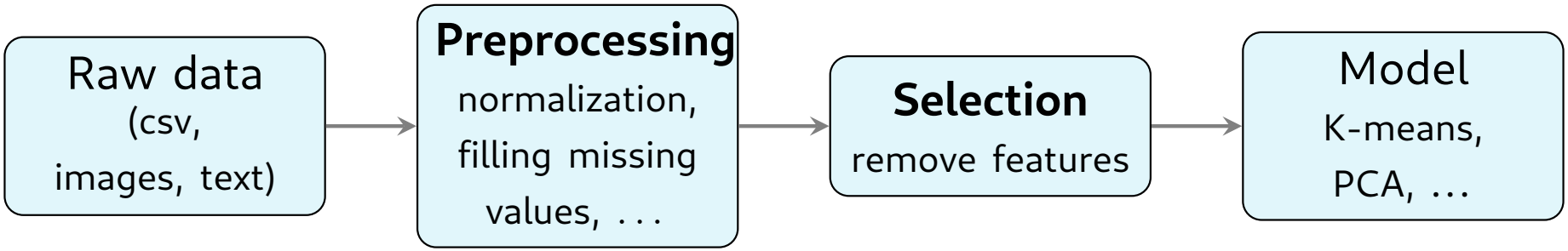
- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

**"Garbage in, garbage out"**

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised and supervised** settings.

# Features in standard Machine Learning pipelines




## Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

**"Garbage in, garbage out"**

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised and supervised** settings.

# Feature preprocessing

**Objective**: change the values of the features, without removing any.

## Classical preprocessing tools

- **Encoding**: turn categories into numbers (`OneHotEncoder`), discretize continuous values,
- **Scaling / standardization**: same mean and variance for all features (`StandardScaler`), or same range (`MinMaxScaler`),
- **Normalization**: put each *sample* on the unit sphere (`Normalizer`),
- **Power / quantile transforms**: make a skewed feature more Gaussian,
- **Manual feature engineering**: build the feature the model cannot guess (ratios, dates $\rightarrow$ day of week).

`https://scikit-learn.org/stable/modules/preprocessing.html`

# Feature selection

**Objective**: remove useless features.
Useful for: figthing the curse of dimensionality, speeding up computations, keeping results interpretable, ...

## Selecting the features

- Remove features with low variance (`VarianceThreshold`): a constant column carries no information,
- Remove redundant features (strongly correlated with another one).

Note: PCA is *not* feature selection! It builds new features as combinations of all of them.

`https://scikit-learn.org/stable/modules/feature_selection.html`

# Plan


1 Clustering

2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

# Conclusion: take-home messages

## Learning without labels

- Goal: discover structure in $X$ (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

## Pitfalls to remember

- The error always decreases with $K$: **never choose $K$ from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

**An unsupervised result is never "correct": it is useful, or not.**

# Conclusion: take-home messages

## Learning without labels

- Goal: discover structure in $X$ (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

## Pitfalls to remember

- The error always decreases with $K$: **never choose $K$ from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

An unsupervised result is never "correct": it is useful, or not.

# Conclusion: take-home messages

## Learning without labels

- Goal: discover structure in $X$ (no labels, hence no "correct answer"),
- **Clustering** groups similar points (e.g. K-means); **dimensionality reduction** describes each point with fewer coordinates (PCA for preprocessing and visualization, t-SNE for visualization only),
- Evaluation without ground truth: internal metrics, labels when they exist, usefulness for the next step.

## Pitfalls to remember

- The error always decreases with $K$: **never choose $K$ from the loss alone**,
- Distances in a t-SNE plot are **not meaningful**,
- Distances and variance depend on the scale of the features: **prepare your data**.

**An unsupervised result is never "correct": it is useful, or not.**

# Plan


1 Clustering

2 Dimensionality reduction

3 Evaluating unsupervised models

4 A note on data cleaning

5 Conclusion

6 Lab session

# Lab Session 3: Unsupervised Learning

## Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Reduce the dimension and visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/`

## Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.

# Lab Session 3: Unsupervised Learning

## Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Reduce the dimension and visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/`

## Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
$\Rightarrow$ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.